Storage Management

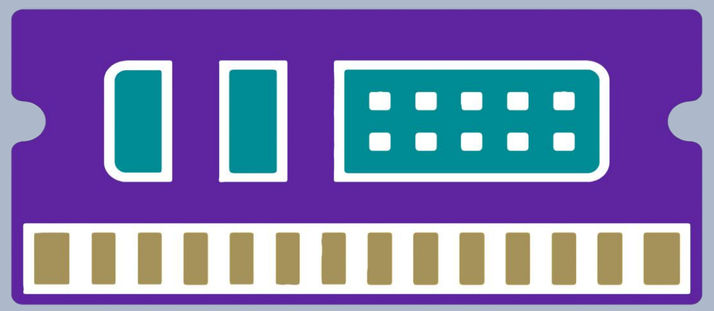
Storage Technologies

Storage Technologies

File I/O in C++

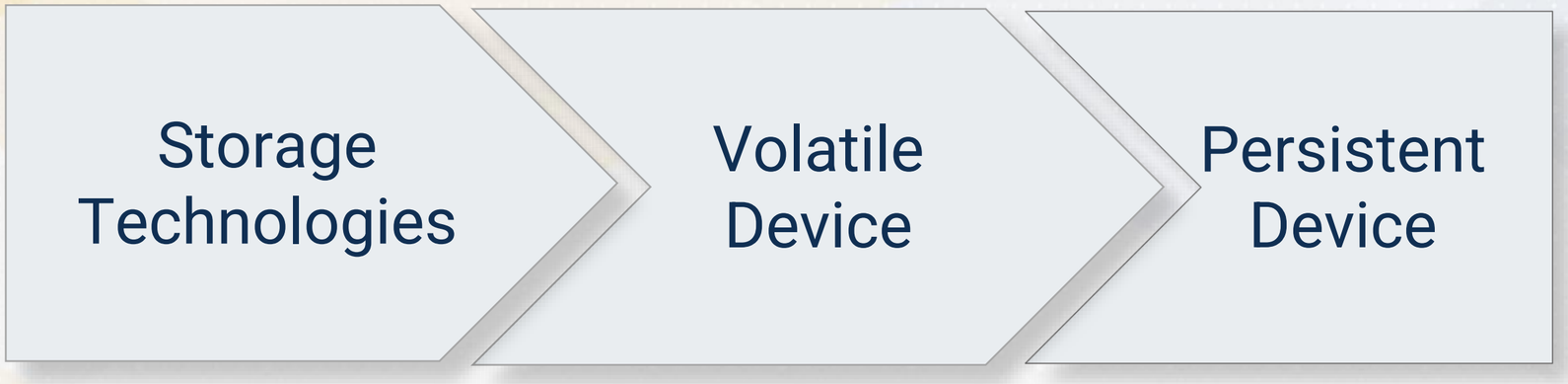
Storage Technologies

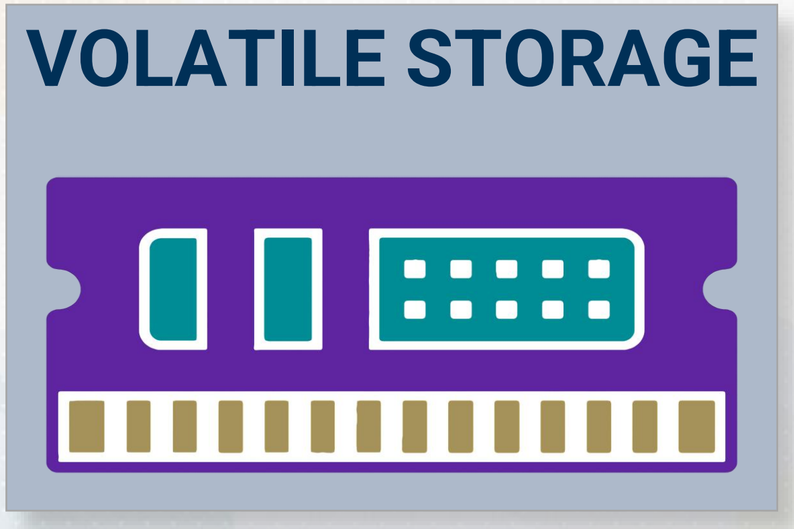
VOLATILE STORAGE



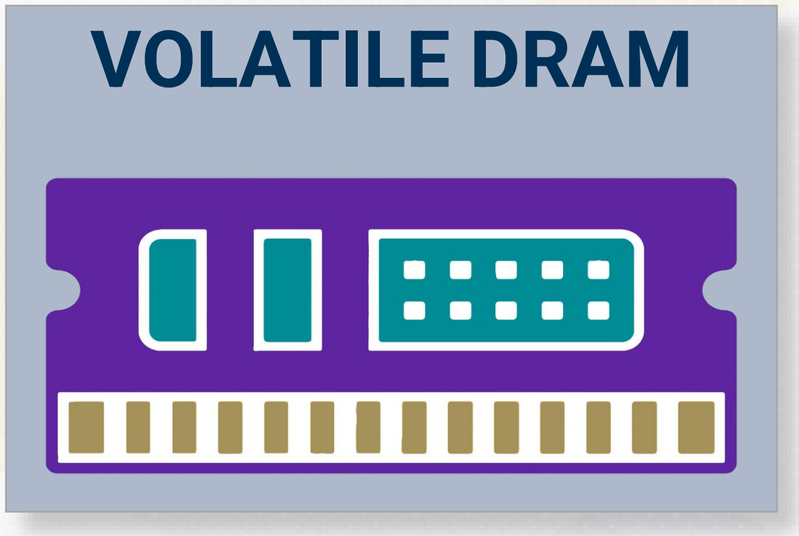
PERSISTENT STORAGE

Storage Technologies

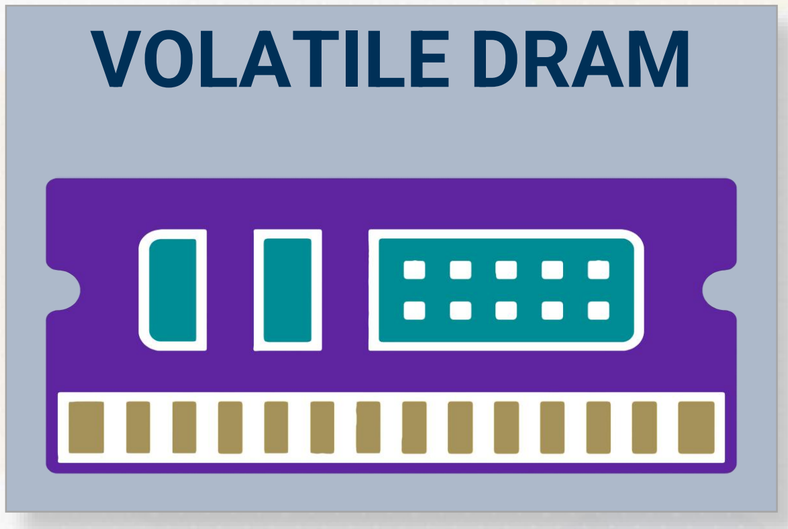




Volatile Storage

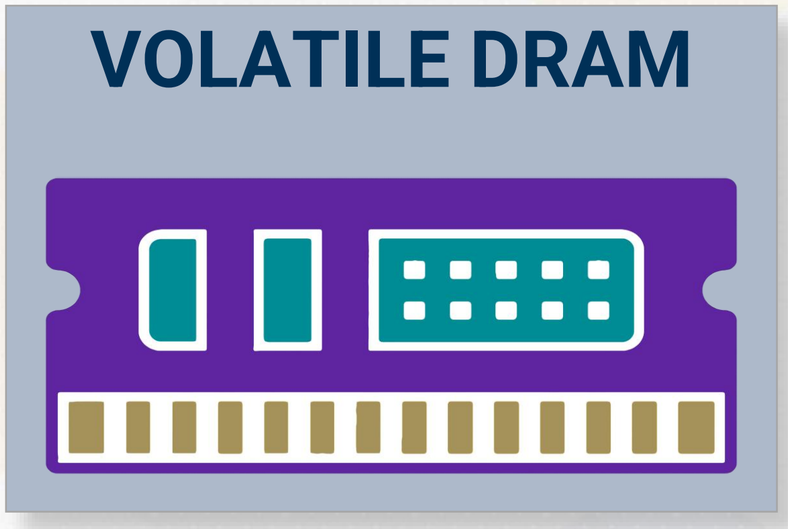


Volatile Storage



Dynamic Random-Access Memory

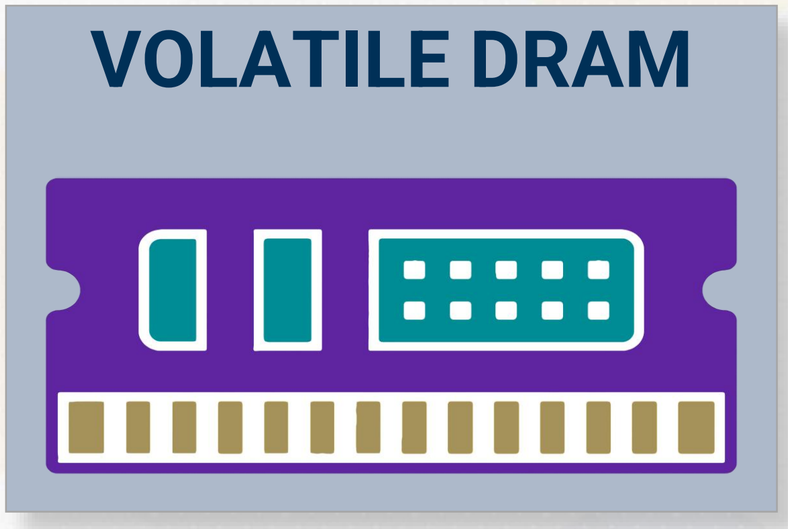
Volatile Storage



Dynamic Random-Access Memory

Cache Frequently Accessed Data

Volatile Storage



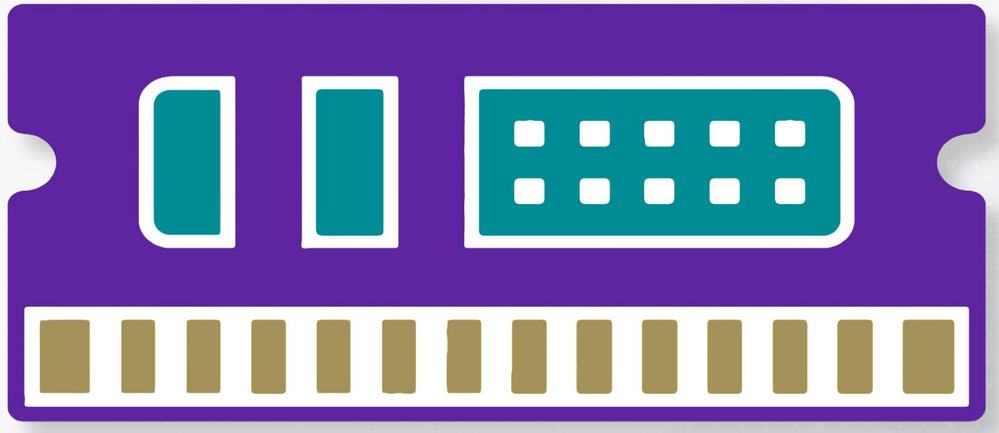
Dynamic Random-Access Memory

Cache Frequently Accessed Data

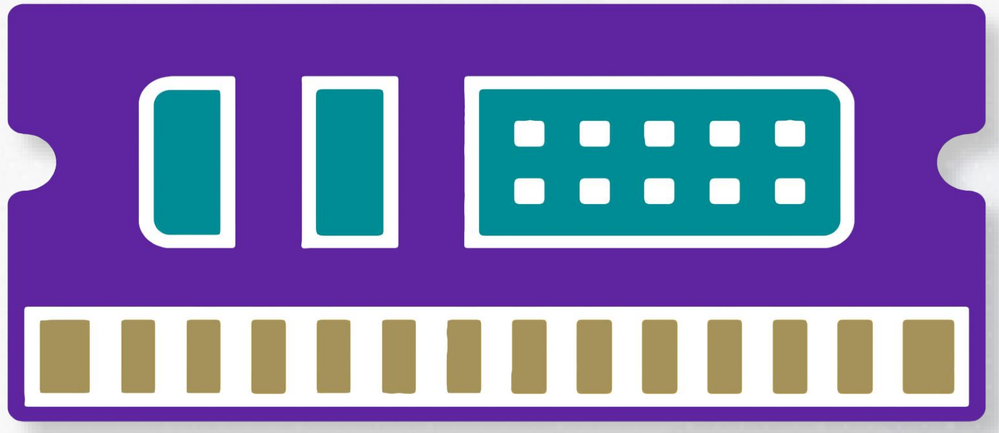
Data Lost Without Power

Persistent Storage

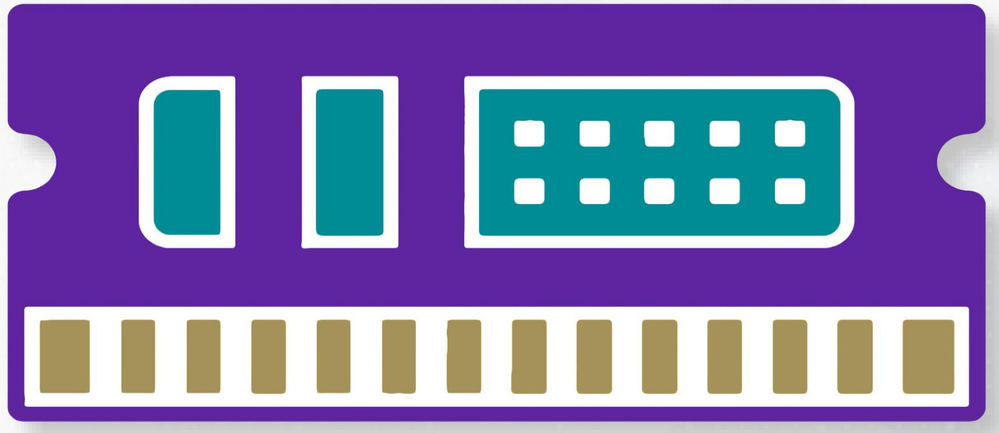
Data Lifecycle: Stage One (DRAM)



Data Lifecycle: Stage One (DRAM)

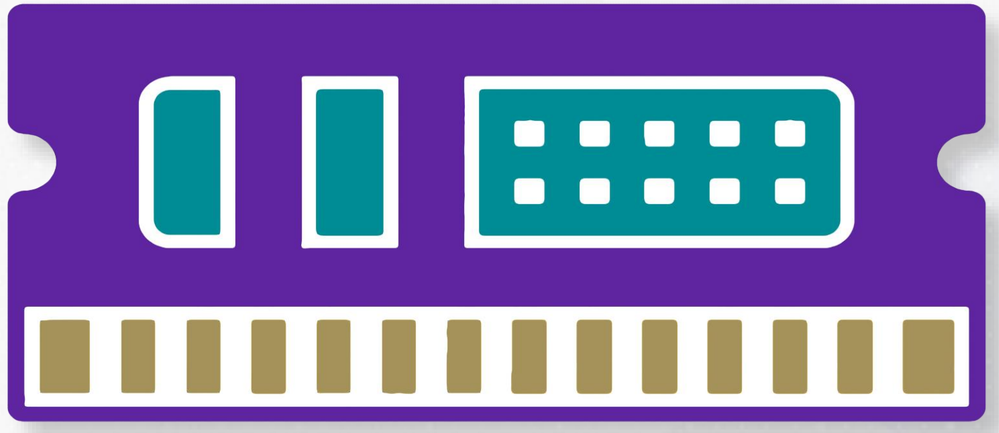


Data Lifecycle: Stage One (DRAM)



"Hot Data"

Data Lifecycle: Stage One (DRAM)



“Hot Data”

Fast Queries and Updates

Data Lifecycle: Stage Two (Disk)

Data Lifecycle: Stage Two (Disk)

Data Lifecycle: Stage Two (Disk)

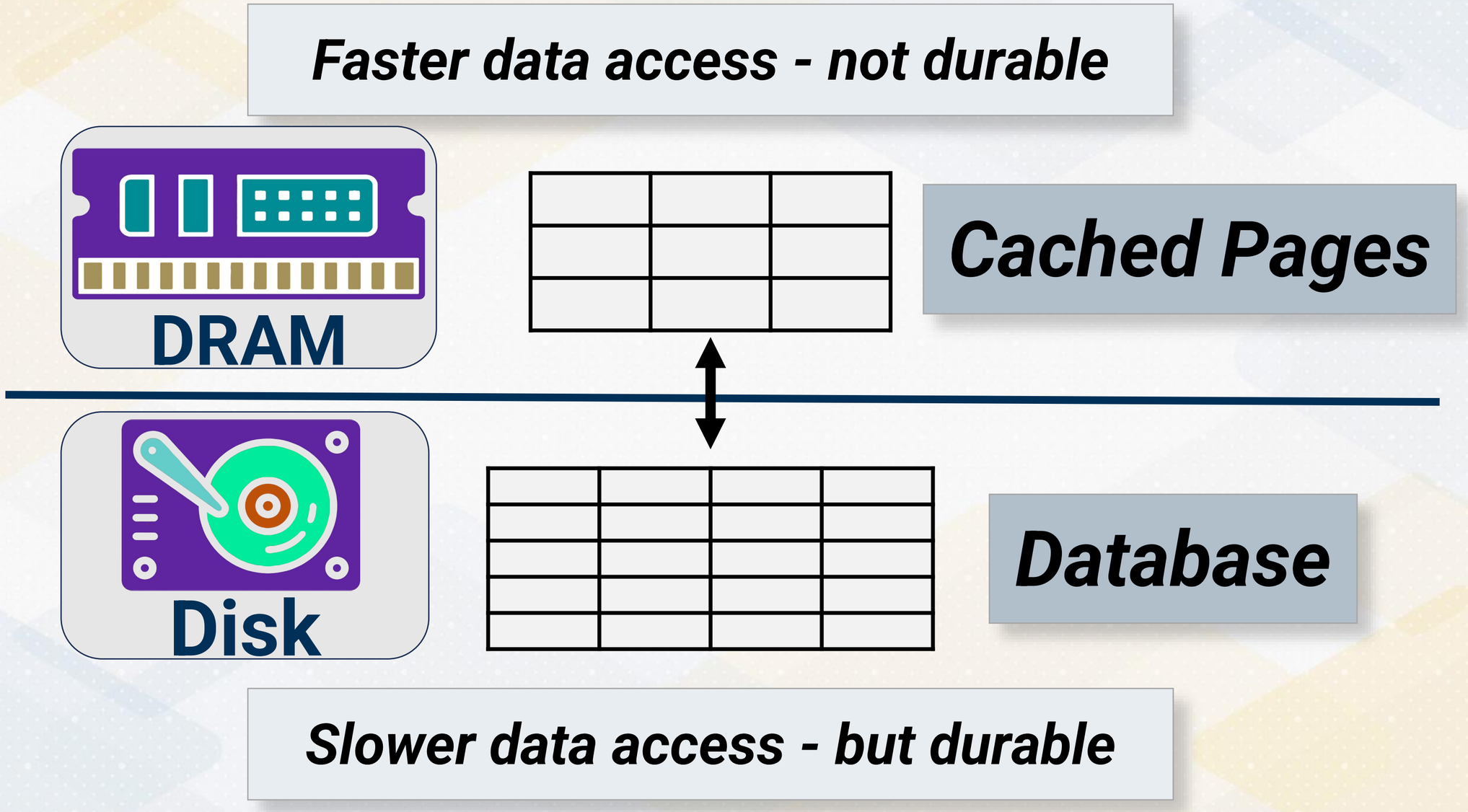
Durability

Data Lifecycle: Stage Two (Disk)

Durability

Data is safe

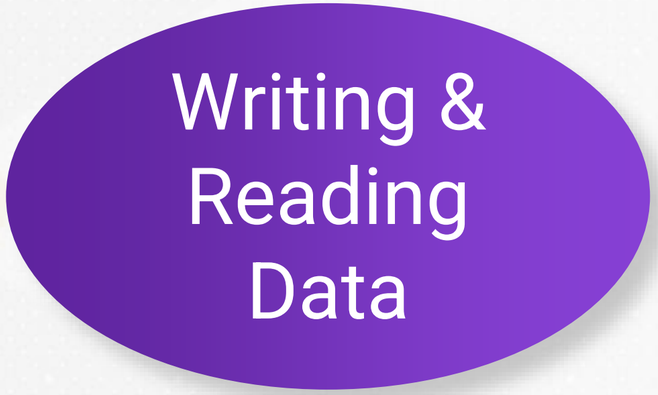
Data Lifecycle



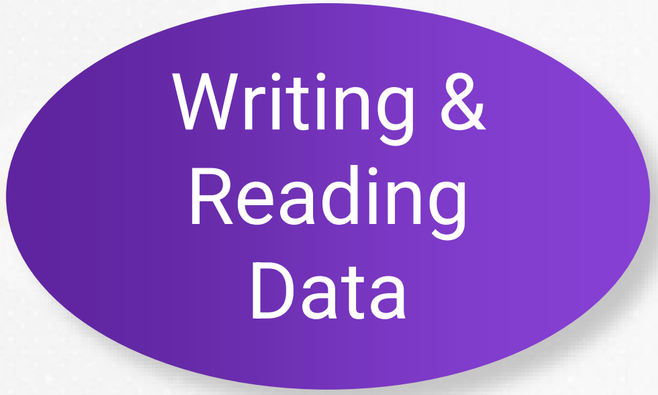
FILE I/O IN C++

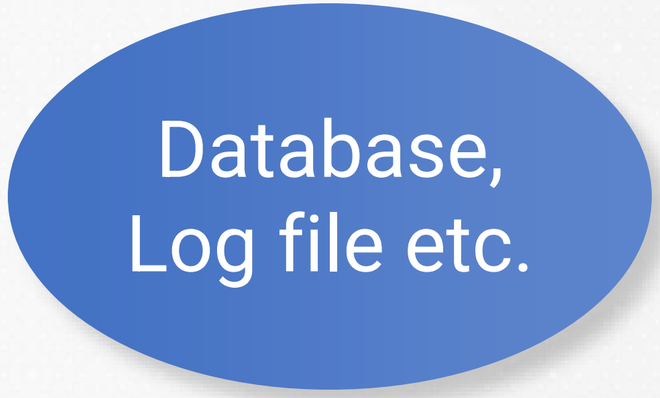
File I/O in C++

File I/O in C++



File I/O in C++





File I/O in C++

buzzDB

```
int main() {
    BuzzDB db;
    // Import data from output.txt file
    std::ifstream inputFile("output.txt");
    int field1, field2;
    // Read pairs of integers from the file
    // and insert them into BuzzDB
    while (inputFile >> field1 >> field2) {
        db.insert(field1, field2);
    }
    // Perform aggregation query on the imported data
    db.selectGroupBySum();
}
```

File I/O in C++

```
std::ifstream inputFile("output.txt"); // Attempt to open file



if (!inputFile) {
    std::cerr << "Unable to open file" << std::endl; // Error handling
    return 1;
}
```

File I/O in C++

Verify File Availability

```
std::ifstream inputFile("output.txt"); // Attempt to open file



if (!inputFile) {
    std::cerr << "Unable to open file" << std::endl; // Error handling
    return 1;
}
```

File I/O in C++

Verify File Availability

Use std::cerr Stream

```
std::ifstream inputFile("output.txt"); // Attempt to open file


if (!inputFile) {
    std::cerr << "Unable to open file" << std::endl; // Error handling
    return 1;
}
```

File I/O in C++

Verify File Availability

Use std::cerr Stream

Avoids Data Disruption

```
std::ifstream inputFile("output.txt"); // Attempt to open file


if (!inputFile) {
    std::cerr << "Unable to open file" << std::endl; // Error handling
    return 1;
}
```

Query Performance in C++

C++ Chrono Library

```
// Get the start time
auto start = std::chrono::high_resolution_clock::now();

BuzzDB db;
...
db.selectGroupBySum();

// Get the end time
auto end = std::chrono::high_resolution_clock::now();
// Calculate the difference between end and start times
std::chrono::duration<double> elapsed = end - start;
// Output the elapsed time in seconds
std::cout << "Elapsed time: " << elapsed.count() << " seconds" << std::endl;
```

Why does timing matter?

Why does timing matter?

Optimize Database Performance

Why does timing matter?

Optimize Database Performance

Comparative Benchmarking

Storage Technologies

Storage Technologies

File I/O in C++

Tuples and Fields

Field Class

Field Class

Constructor and Destructor

Field Class

Constructor and Destructor

New and Delete Operators

Tuple Class

```
class Tuple {
public:
    // Identifier field
    int key;
    // Actual data field
    int value;
};
```

Tuple Class

Integer Key-Value Pairs

```
class Tuple {
public:
    // Identifier field
    int key;
    // Actual data field
    int value;
};
```

Tuple Class

Integer Key-Value Pairs

Floating Point Numbers

```
class Tuple {
public:
    // Identifier field
    int key;
    // Actual data field
    int value;
};
```

Tuple Class

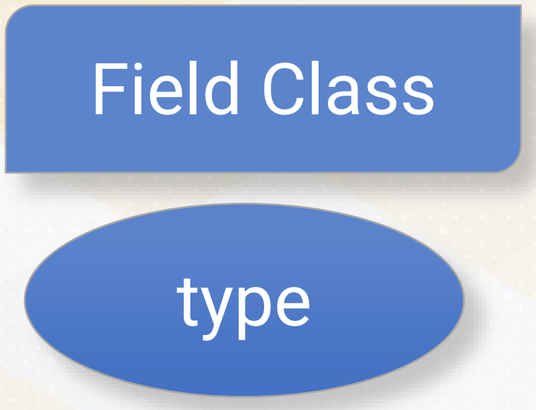
Integer Key-Value Pairs	Floating Point Numbers	Strings

```
class Tuple {
public:
    // Identifier field
    int key;
    // Actual data field
    int value;
};
```

Field Class

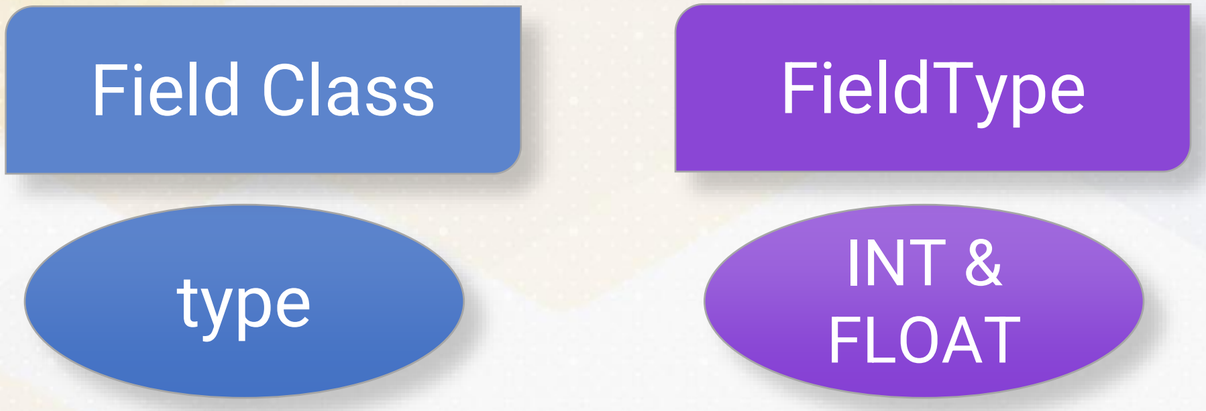
```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class



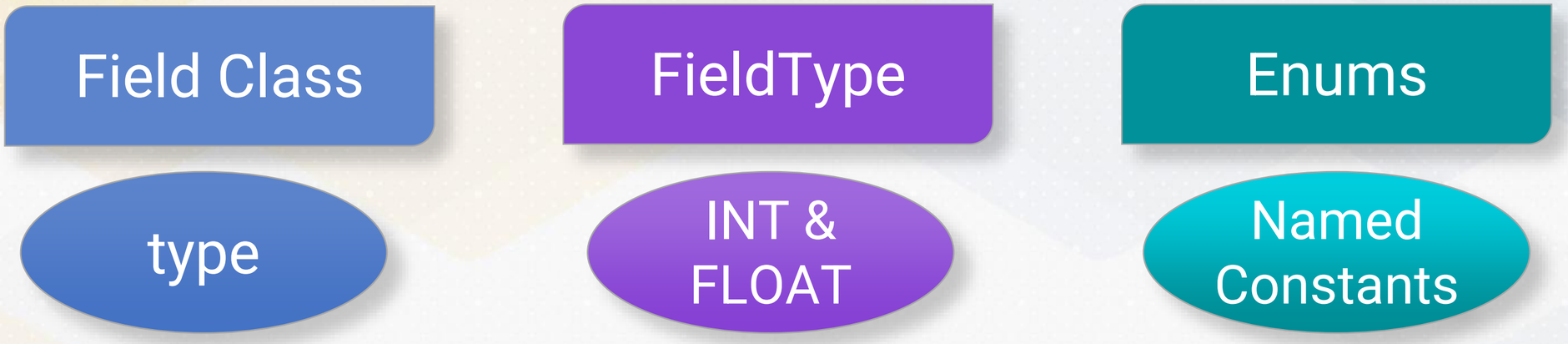

```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class




```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class

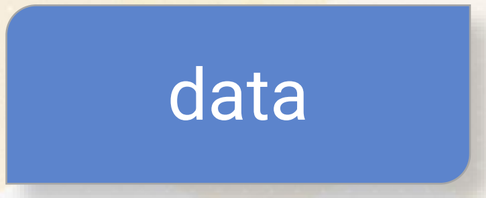



```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class

```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class

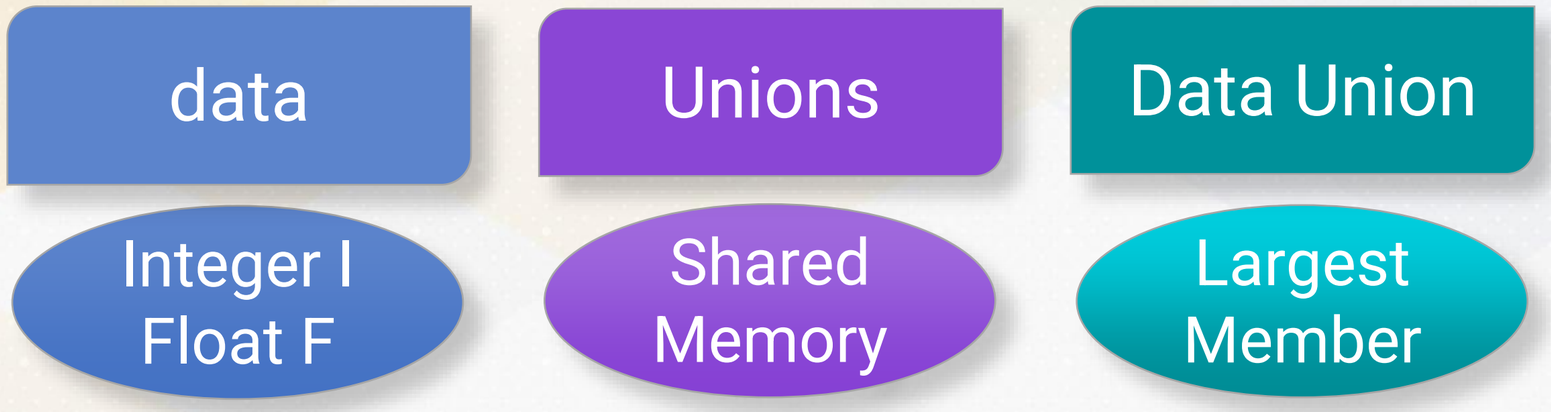



```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class

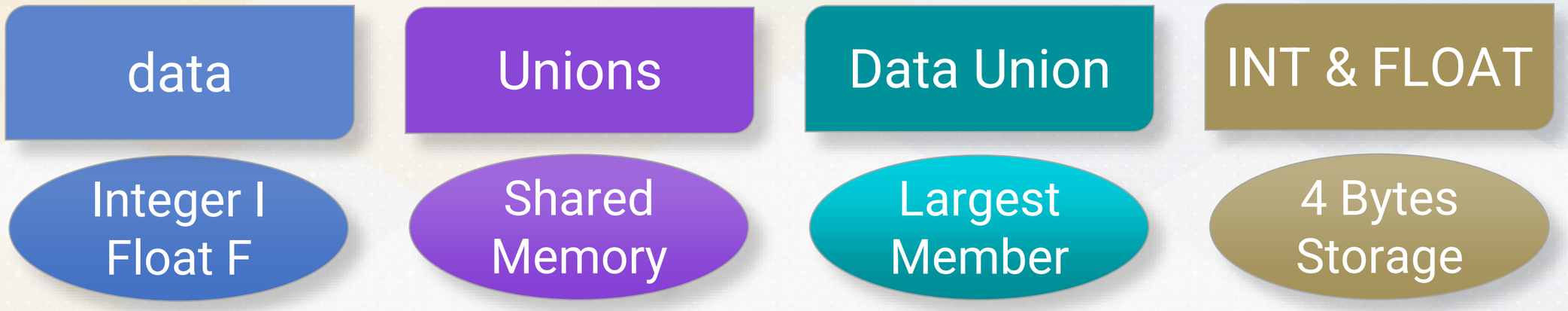

```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class




```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class




```
enum FieldType { INT, FLOAT }

class Field {
    public :
    FieldType type;
    union {
        int i;
        float f;
    } data;
};
```

Field Class

```
class Field {
   public :
   Field(int i) : type(INT), data{i} {}
   Field(float f) : type(FLOAT), data{f} {}
   void print() const {
      switch (type) {
      case INT: std::cout << data.i; break;
      case FLOAT: std::cout << data.f; break;
   }
  }
};
```

Field Class

Integer
Value INT

```
class Field {
   public :
   Field(int i) : type(INT), data{i} {}
   Field(float f) : type(FLOAT), data{f} {}
   void print() const {
      switch (type) {
      case INT: std::cout << data.i; break;
      case FLOAT: std::cout << data.f; break;
   }
  }
};
```

Field Class

Integer Value INT

Float Value FLOAT

```
class Field {
    public :
    Field(int i) : type(INT), data{i} {}
    Field(float f) : type(FLOAT), data{f} {}
    void print() const {
        switch (type) {
        case INT: std::cout << data.i; break;
        case FLOAT: std::cout << data.f; break;
    }
  }
};
```

Field Class

Integer Value INT

Float Value FLOAT

Constructor Overloading

```
class Field {
    public :
    Field(int i) : type(INT), data{i} {}
    Field(float f) : type(FLOAT), data{f} {}
    void print() const {
        switch (type) {
        case INT: std::cout << data.i; break;
        case FLOAT: std::cout << data.f; break;
    }
  }
};
```

Field Class

Integer Value INT

Float Value FLOAT

Constructor Overloading

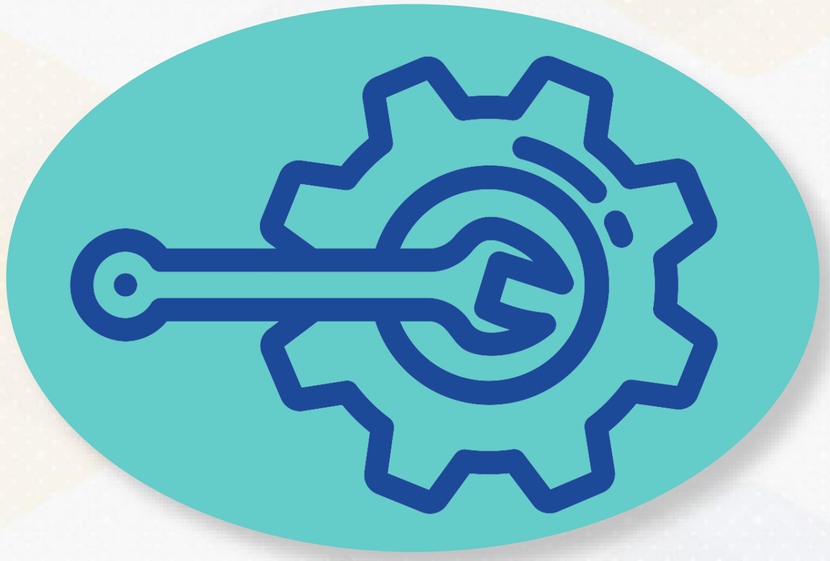
Print Method

```
class Field {
   public :
   Field(int i) : type(INT), data{i} {}
   Field(float f) : type(FLOAT), data{f} {}
   void print() const {
      switch (type) {
      case INT: std::cout << data.i; break;
      case FLOAT: std::cout << data.f; break;
   }
  }
};
```

C++ Constructors and Destructors

Constructors in C++

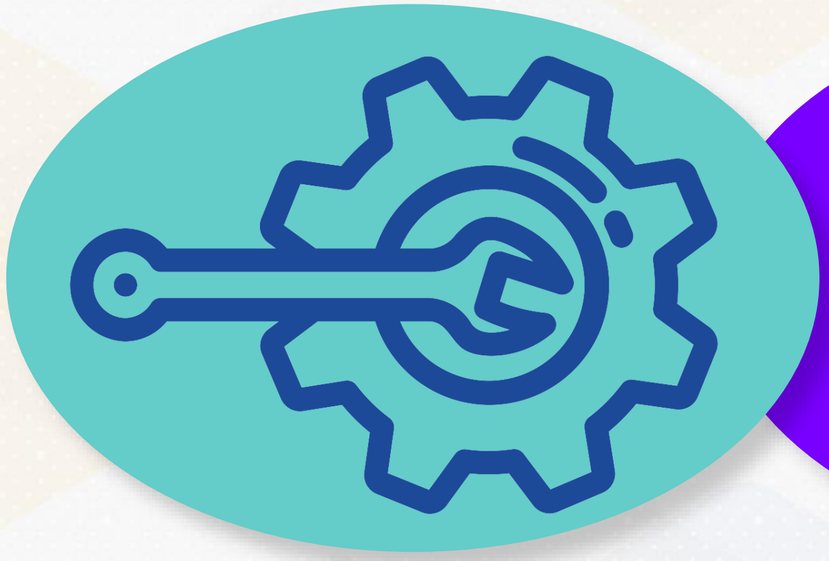
Constructors in C++



Constructors in C++

Construct
Objects with
Specific Values

Constructors in C++



Construct Objects with Specific Values

Initialize Integer or Float Data

Generalized Tuple Class

```
class Tuple {
    std::vector<Field> fields;
    public:
    void addField(const Field &field) {
        fields.push_back(field);
    }
    void print() const {
        for (const auto &field : fields) {
            field.print();
            std::cout << " ";
    }
  }
};
```

Constructing Generalized Tuples

```
void BuzzDB::insert(int key, int value) {
    Tuple newTuple;
    Field key_field = Field(key);
    Field value_field = Field(value);
    float float_val = 132.04;
    Field float_field = Field(float_val);

    newTuple.addField(key_field);
    newTuple.addField(value_field);
    newTuple.addField(float_field);

    table.push_back(newTuple);
    index[key].push_back(value);
}
```

Further Generalization to Strings

```
enum FieldType { INT, FLOAT, STRING }
class Field {
   public :
   FieldType type;
   union {
      int i;
      float f;
      char *s; // string
   } data;
  ...
}
```

Further Generalization to Strings

FieldType enum now includes STRING

```
enum FieldType { INT, FLOAT, STRING }
class Field {
   public :
   FieldType type;
   union {
      int i;
      float f;
      char *s; // string
   } data;
  ...
}
```

More Generalization to Strings

More Generalization to Strings

Polymorphic Container

More Generalization to Strings

Polymorphic Container

Field Object

Integer Form

Float Form

String Form

More Generalization to Strings

Polymorphic Container

Field Object

Integer Form

Float Form

String Form

DATABASE = 8 Letters

POLYMORPHISM = 12 Letters

More Generalization to Strings

```
Field(const std::string &s)
    : type(STRING) {
    data.s = new char[s.size() + 1];
    std::copy(s.begin(), s.end(), data.s);
    data.s[s.size()] = '\0'; // Null-terminate the string
}
```

More Generalization to Strings

Dynamic Memory Allocation

```
Field(const std::string &s)
    : type(STRING) {
    data.s = new char[s.size() + 1];
    std::copy(s.begin(), s.end(), data.s);
    data.s[s.size()] = '\0'; // Null-terminate the string
}
```

More Generalization to Strings

Dynamic Memory Allocation

String parameter S

```
Field(const std::string &s)
    : type(STRING) {
    data.s = new char[s.size() + 1];
    std::copy(s.begin(), s.end(), data.s);
    data.s[s.size()] = '\0'; // Null-terminate the string
}
```

More Generalization to Strings

Dynamic Memory Allocation

String parameter S

Field object to String

```
Field(const std::string &s)
    : type(STRING) {
    data.s = new char[s.size() + 1];
    std::copy(s.begin(), s.end(), data.s);
    data.s[s.size()] = '\0'; // Null-terminate the string
}
```

More Generalization to Strings

Dynamic Memory Allocation

String parameter S

Field object to String

Size of S + 1

```
Field(const std::string &s)
    : type(STRING) {
    data.s = new char[s.size() + 1];
    std::copy(s.begin(), s.end(), data.s);
    data.s[s.size()] = '\0'; // Null-terminate the string
}
```

More Generalization to Strings

Dynamic Memory Allocation

String parameter S

Field object to String

Size of S + 1

Copy into data.s

```
Field(const std::string &s)
    : type(STRING) {
    data.s = new char[s.size() + 1];
    std::copy(s.begin(), s.end(), data.s);
    data.s[s.size()] = '\0'; // Null-terminate the string
}
```

New Operator


```
data.s = new char[s.size() + 1];
```

New Operator

Dynamic Memory Allocation

```
data.s = new char[s.size() + 1];
```

New Operator

Dynamic Memory Allocation

Heap-Allocated Variable

```
data.s = new char[s.size() + 1];
```

New Operator

Dynamic Memory Allocation

Heap-Allocated Variable

Explicit Variable Destruction

```
data.s = new char[s.size() + 1];
```

Destructors in C++


```
Field::~Field() {
    if (type == STRING && data.s != nullptr) {
        delete[] data.s;
  }
}
```

Destructors in C++

Field Class Destructor

```
Field::~Field() {
    if (type == STRING && data.s != nullptr) {
        delete[] data.s;
  }
}
```

Destructors in C++

Field Class Destructor

Cleans Memory

```
Field::~Field() {
    if (type == STRING && data.s != nullptr) {
        delete[] data.s;
  }
}
```

Destructors in C++

Field Class Destructor

Cleans Memory

data.s ≠ nullptr

```
Field::~Field() {
    if (type == STRING && data.s != nullptr) {
        delete[] data.s;
  }
}
```

Delete Operator


```
delete ptr;   // Frees memory allocated for a single integer
delete[] arr; // Frees memory allocated for an array of integers
```

Delete Operator


```
delete ptr;   // Frees memory allocated for a single integer
delete[] arr; // Frees memory allocated for an array of integers
```

Delete Operator

Free
Heap
Memory

Matching
Delete
Operator

```
delete ptr;   // Frees memory allocated for a single integer
delete[] arr; // Frees memory allocated for an array of integers
```

Delete Operator

Free Heap Memory

Matching Delete Operator

Avoid Memory Leaks

```
delete ptr;   // Frees memory allocated for a single integer
delete[] arr; // Frees memory allocated for an array of integers
```

Field Class

Field Class

Constructor and Destructor

Field Class

Constructor and Destructor

New and Delete Operators

Smart Pointers

Raw Pointers

Raw Pointers

Smart Pointers

Raw Pointers

Field Class ✚ Raw Character Pointer

```
enum FieldType { INT, FLOAT, STRING }
class Field {
    public :
    FieldType type;
    union {
            int i;
            float f;
            char *s;
    } data;
    ...
};
```

Perils of Raw Pointers

Four Important Problems

Perils of Raw Pointers

Four Important Problems

Memory Leaks

Perils of Raw Pointers

Four Important Problems

- Memory Leaks
- Dangling Pointers

Perils of Raw Pointers

Four Important Problems

- Memory Leaks
- Dangling Pointers
- Double-Free Errors

Perils of Raw Pointers

Four Important Problems

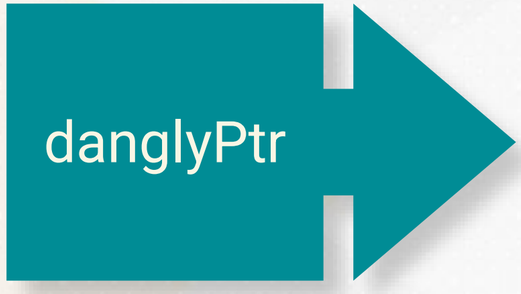
- Memory Leaks
- Dangling Pointers
- Double-Free Errors
- Ownership Ambiguity

Perils of Raw Pointers: Memory Leaks

Explicit Deletion Required

```
void createMemoryLeak() {
    int *leakyPtr = new int(42); // Allocation
    // Forgot to delete leakyPtr
    // Memory allocated to leakyPtr is never freed
}
```

Perils of Raw Pointers: Dangling Pointers

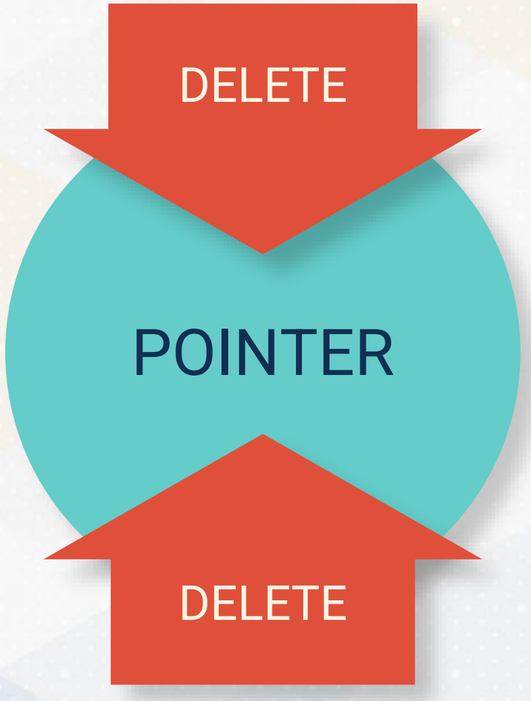



```
void createDanglingPointer() {
    int *danglyPtr = new int(42)
   delete danglyPtr
   // Deallocate memory
   // danglyPtr now becomes a dangling pointer
   std::cout << *danglyPtr
   // Undefined behavior, potentially a crash
}
```

Perils of Raw Pointers: Dangling Pointers


```
void createDanglingPointer() {
    int *danglyPtr = new int(42)
   delete danglyPtr
   // Deallocate memory
   // danglyPtr now becomes a dangling pointer
   std::cout << *danglyPtr
   // Undefined behavior, potentially a crash
}
```

Perils of Raw Pointers: Double-Free Errors




```
void createDoubleFree() {
    int *doubleTroublePtr = new int(42);
    // Correct deletion
    delete doubleTroublePtr;
    // Double free, leads to runtime error
    delete doubleTroublePtr;
}
```

Perils of Raw Pointers

```
int *function1(int *ptr) {
  // It's unclear if the function should delete ptr
  // Function logic here...
  // Should the caller delete the returned pointer?
  return new int(55);
}
void function2() {
    int *myPtr = new int(42);
    int *newPtr = function1(myPtr);
    // Who owns myPtr and newPtr?
    // Who is responsible for deleting them?
}
```

Perils of Raw Pointers

No Clear Ownership

```
int *function1(int *ptr) {
  // It's unclear if the function should delete ptr
  // Function logic here...
  // Should the caller delete the returned pointer?
  return new int(55);
}
void function2() {
    int *myPtr = new int(42);
    int *newPtr = function1(myPtr);
    // Who owns myPtr and newPtr?
    // Who is responsible for deleting them?
}
```

Perils of Raw Pointers

No Clear Ownership

Unclear Responsibility

```
int *function1(int *ptr) {
  // It's unclear if the function should delete ptr
  // Function logic here...
  // Should the caller delete the returned pointer?
  return new int(55);
}
void function2() {
    int *myPtr = new int(42);
    int *newPtr = function1(myPtr);
    // Who owns myPtr and newPtr?
    // Who is responsible for deleting them?
}
```

Perils of Raw Pointers

No Clear Ownership

Unclear Responsibility

Complex Memory Management

```
int *function1(int *ptr) {
  // It's unclear if the function should delete ptr
  // Function logic here...
  // Should the caller delete the returned pointer?
  return new int(55);
}
void function2() {
    int *myPtr = new int(42);
    int *newPtr = function1(myPtr);
    // Who owns myPtr and newPtr?
    // Who is responsible for deleting them?
}
```

Smart Pointers

Smart Pointers

std::unique_ptr

Smart Pointers

- std::unique_ptr
- Code Safety
- Readability
- Maintainability

Smart Pointers: Avoid Memory Leaks

```
#include <memory>
void createMemoryLeak() {
    std::unique_ptr<int> safePtr =
    std::make_unique<int>(42);
    // Automatic memory management
    // No need to manually delete
    // Memory is automatically freed
    // when safePtr goes out of scope
}
```

Smart Pointers: Avoid Memory Leaks

```
#include <memory>
void createMemoryLeak() {
    std::unique_ptr<int> safePtr =
    std::make_unique<int>(42);
    // Automatic memory management
    // No need to manually delete
    // Memory is automatically freed
    // when safePtr goes out of scope
}
```

std::unique_ptr

Smart Pointers: Avoid Memory Leaks

```
#include <memory>
void createMemoryLeak() {
    std::unique_ptr<int> safePtr =
    std::make_unique<int>(42);
    // Automatic memory management
    // No need to manually delete
    // Memory is automatically freed
    // when safePtr goes out of scope
}
```

std::unique_ptr

Automatic
Memory Management

Smart Pointers: Avoid Memory Leaks

```cpp
#include <memory>
void createMemoryLeak() {
    std::unique_ptr<int> safePtr =
    std::make_unique<int>(42);
    // Automatic memory management
    // No need to manually delete
    // Memory is automatically freed
    // when safePtr goes out of scope
}
```

std::unique_ptr

Automatic
Memory Management

Prevent
Memory Leaks

Smart Pointers: Avoid Dangling Pointers

After calling reset(), safePtr safely releases its ownership and avoids becoming a dangling pointer by setting itself to nullptr

```
void createDanglingPointer() {
    std::unique_ptr<int> safePtr = std::make_unique<int>(42);
    safePtr.reset(); // Correctly frees memory and sets pointer to nullptr
    // safePtr is now nullptr, preventing access to freed memory
}
```

Smart Pointers: Avoid Doube-Free Errors

std::unique_ptr enforces unique ownership of the managed object, meaning that the memory is freed exactly once when the pointer goes out of scope or is reset.

Prevents double-free errors

```
void createDoubleFree() {
    std::unique_ptr<int> safePtr =
    std::make_unique<int>(42);
    // safePtr goes out of scope
    // and automatically deletes the managed object
    // No risk of double free errors
    // as unique_ptr ensures single ownership
    // and controlled deletion
}
```

Smart Pointers: Avoid Ownership Ambiguity

unique_ptr clarifies ownership through its ownership model

```
std::unique_ptr<int> function1(std::unique_ptr<int> ptr) {
    // Ownership is clearly transferred with unique_ptr, no ambiguity
    return std::make_unique<int>(55);
    // Returning a new unique_ptr transfers ownership back to the caller
}
void function2() {
    std::unique_ptr<int> myPtr = std::make_unique<int>(42);
    std::unique_ptr<int> newPtr = function1(std::move(myPtr));
    // Ownership transfer is explicit with std::move,
    // eliminating ambiguity
}
```

Raw Pointers

Raw Pointers

Smart Pointers

Smart Fields

Smart Field and Tuple Classes

Smart Field and Tuple Classes

Copy Semantics in C++

Smart Field and Tuple Classes

Copy Semantics in C++

Heap vs Stack

Smart Pointer in Field

Transition from manual memory management to std::unique_ptr for string

```
class Field {
public:
    int data_i;
    float data_f;
    std::unique_ptr<char[]> data_s;
    size_t data_s_length;
    ...
}
```

Smart Pointer in Field

unique_ptr

Minimize Memory Leak Risk

Destructor Eliminates delete[] Calls

```
Field(const std::string &s)
    : type(STRING) {
    data_s_length = s.size() + 1;
    data_s = std::make_unique<char[]>(data_s_length);
    std::strcpy(data_s.get(), s.c_str());
}
```

Smart Tuple Class

Tuple Class

```
class Tuple {
    std::vector<std::unique_ptr<Field>> fields;

    public :
    void addField(std::unique_ptr<Field> field) {
        fields.push_back(std::move(field))
    }
};
```

unique_ptr cannot be copied in the traditional sense to ensure unique ownership

Smart Tuple Class

Tuple Class

std::vector

```
class Tuple {
    std::vector<std::unique_ptr<Field>> fields;

    public :
    void addField(std::unique_ptr<Field> field) {
        fields.push_back(std::move(field))
    }
};
```

unique_ptr cannot be copied in the traditional sense to ensure unique ownership

Smart Tuple Class

Tuple Class

std::vector

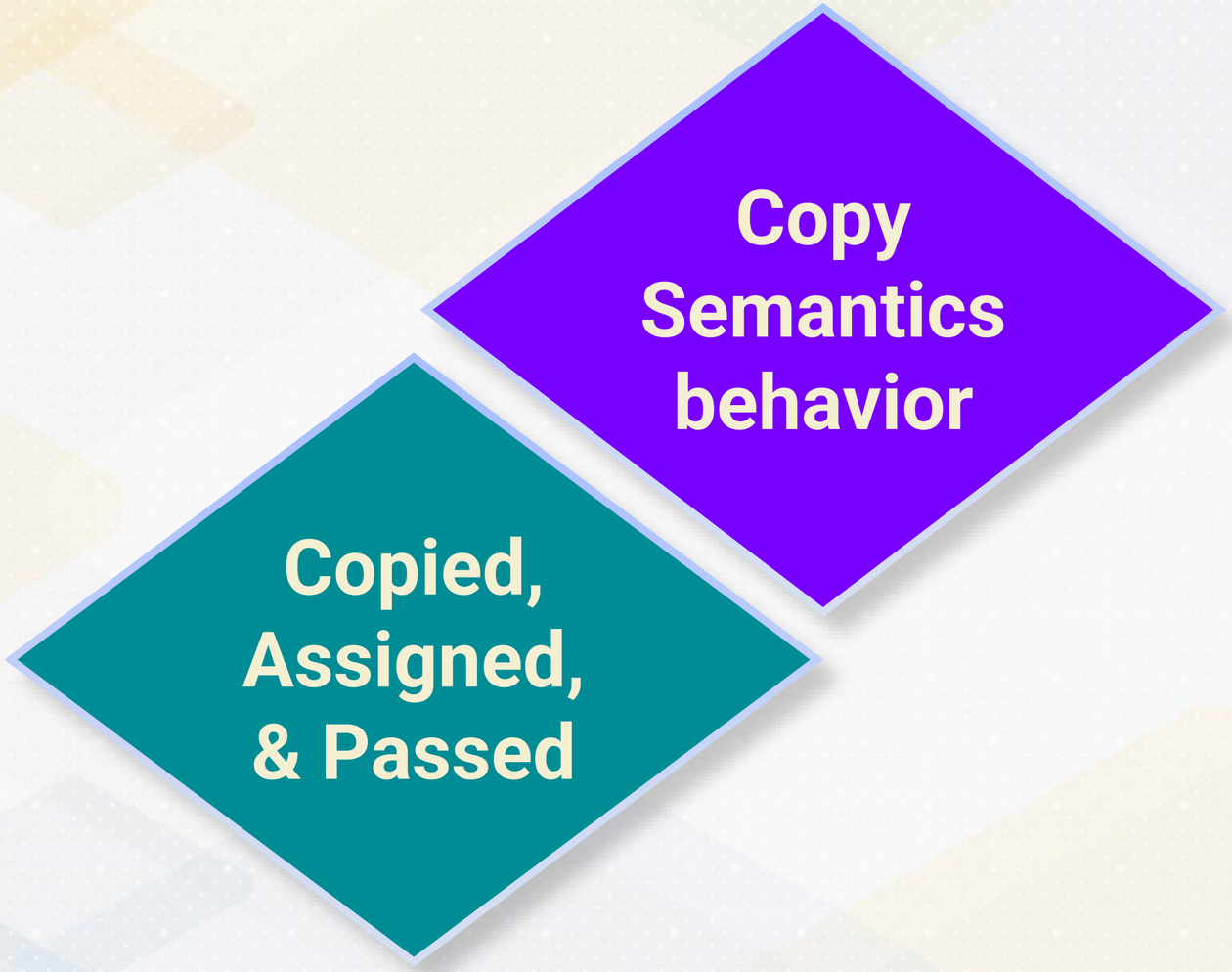
std::unique_ptr<Field>

```
class Tuple {
    std::vector<std::unique_ptr<Field>> fields;

    public :
    void addField(std::unique_ptr<Field> field) {
        fields.push_back(std::move(field))
    }
};
```

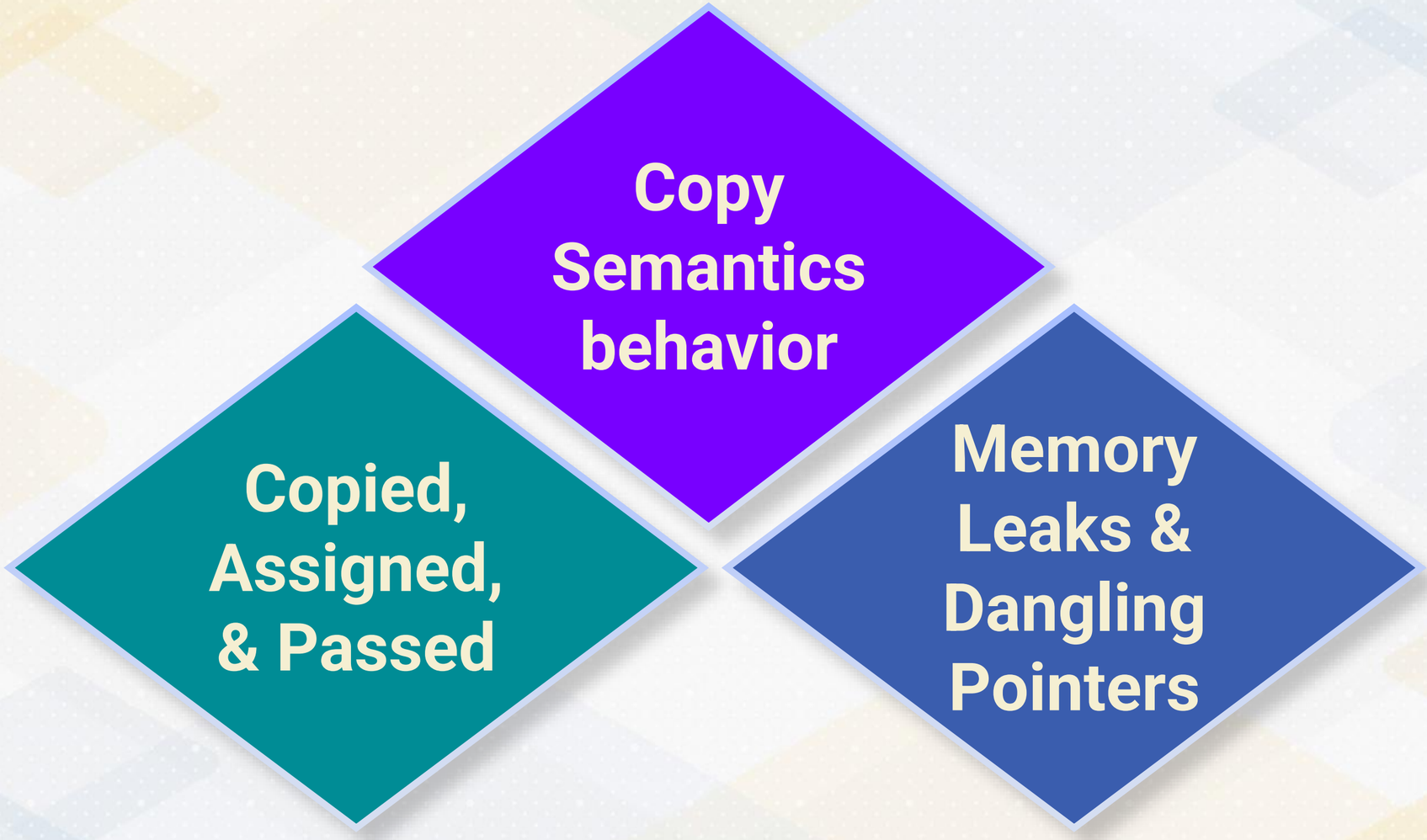
unique_ptr cannot be copied in the traditional sense to ensure unique ownership

Copy Semantics in C++

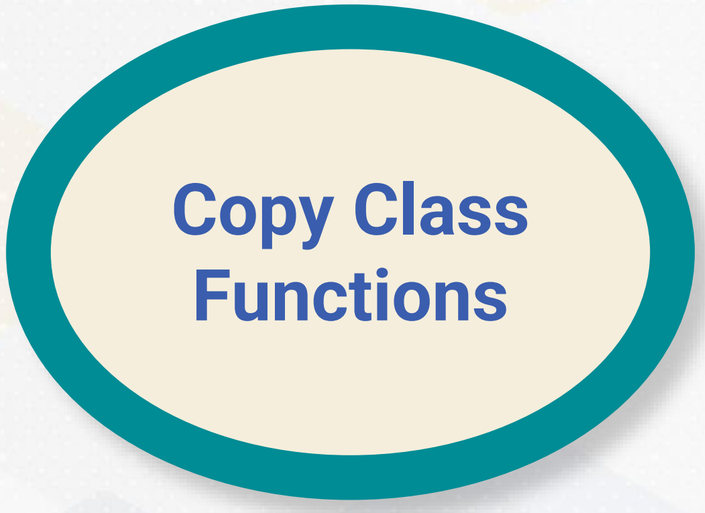
Copy Semantics in C++



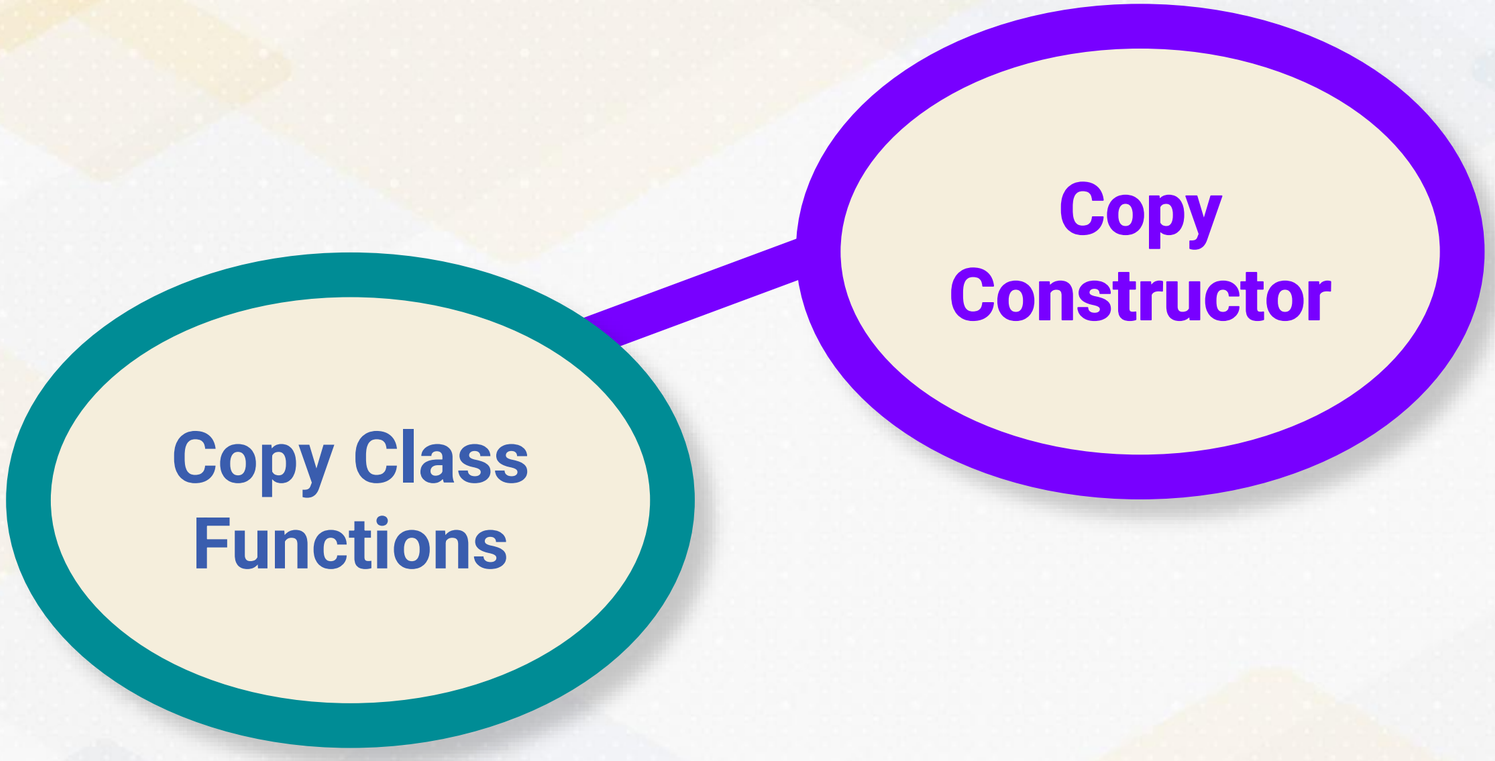
Copy Semantics in C++



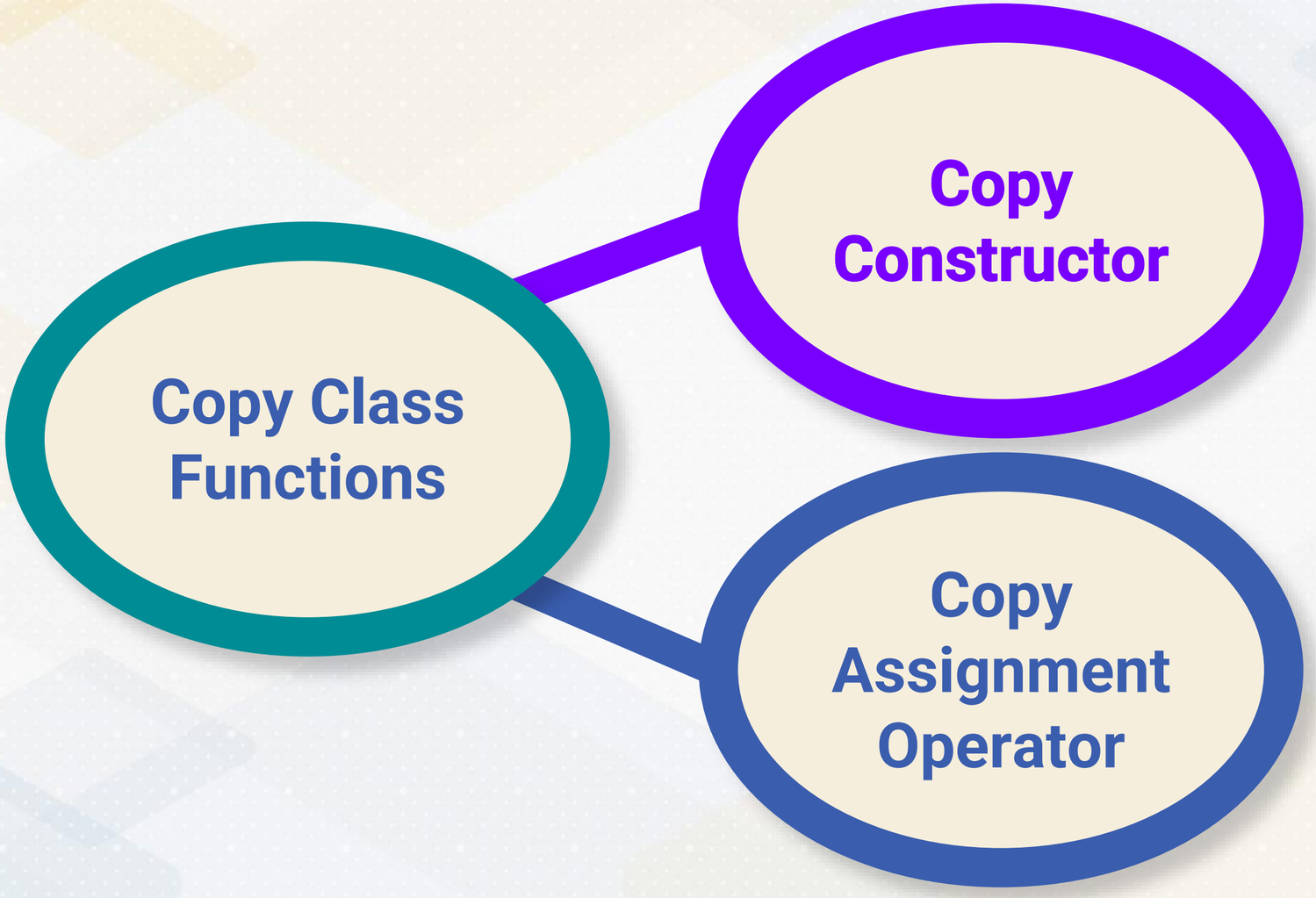
Copy Semantics in C++



Copy Semantics in C++



Copy Semantics in C++



Copy Constructor

```
Field(const Field &other) : type(other.type) {
  switch (type) {
  case STRING:
      data_s_length = other.data_s_length;
      data_s = std::make_unique<char[]>(data_s_length);
      strcpy(data_s.get(), other.data_s.get());
      break;
  }
}
```

Field field2 = field1;

Copy Constructor

```
Field(const Field &other) : type(other.type) {
  switch (type) {
  case STRING:
      data_s_length = other.data_s_length;
      data_s = std::make_unique<char[]>(data_s_length);
      strcpy(data_s.get(), other.data_s.get());
      break;
  }
}
```

Field field2 = field1;

Copy constructor creates new independent copy

Copy Constructor

```
Field(const Field &other) : type(other.type) {
  switch (type) {
  case STRING:
      data_s_length = other.data_s_length;
      data_s = std::make_unique<char[]>(data_s_length);
      strcpy(data_s.get(), other.data_s.get());
      break;
  }
}
```

Field field2 = field1;

Copy constructor creates new independent copy

Deep Copying

Copy Assignment Operator

A class method that assigns the contents of one Field object to another Field object: field1 = field2

```
Field &operator=(const Field &other) {
   if (this != &other) {
      switch (type) {
      case STRING:
         data_s_length = other.data_s_length;
         data_s = std::make_unique<char[]>(data_s_length);
         strcpy(data_s.get(), other.data_s.get());
         break;
   }
 }
 return *this;
}
```

Move Sematics in C++

While unique_ptr cannot be copied, it can be moved to transfer ownership of the managed object from one unique_ptr to another

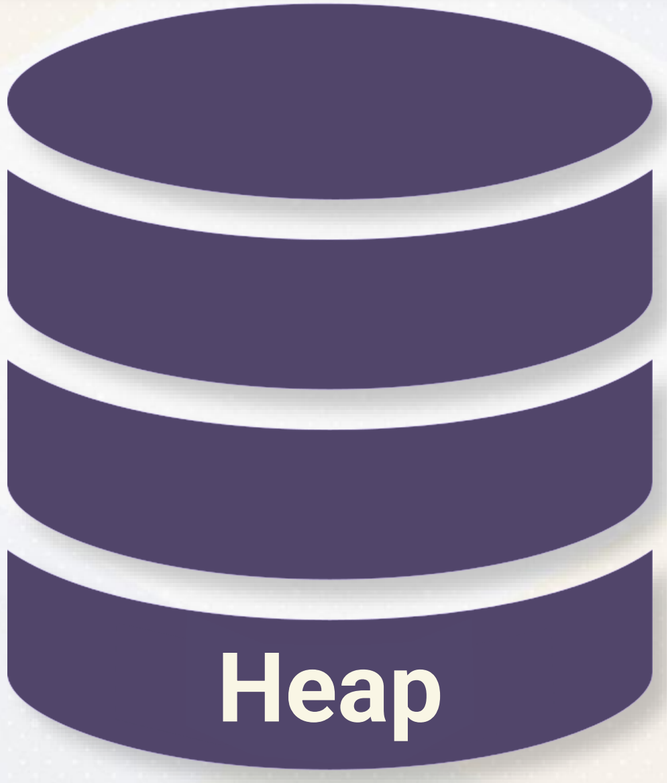
```
newTuple->addField(std::move(key_field))
```

HEAP vs STACK

Heap Allocation

Ideal for objects and data structures whose size cannot be determined at compile time

When you need the data to persist across different parts of the program

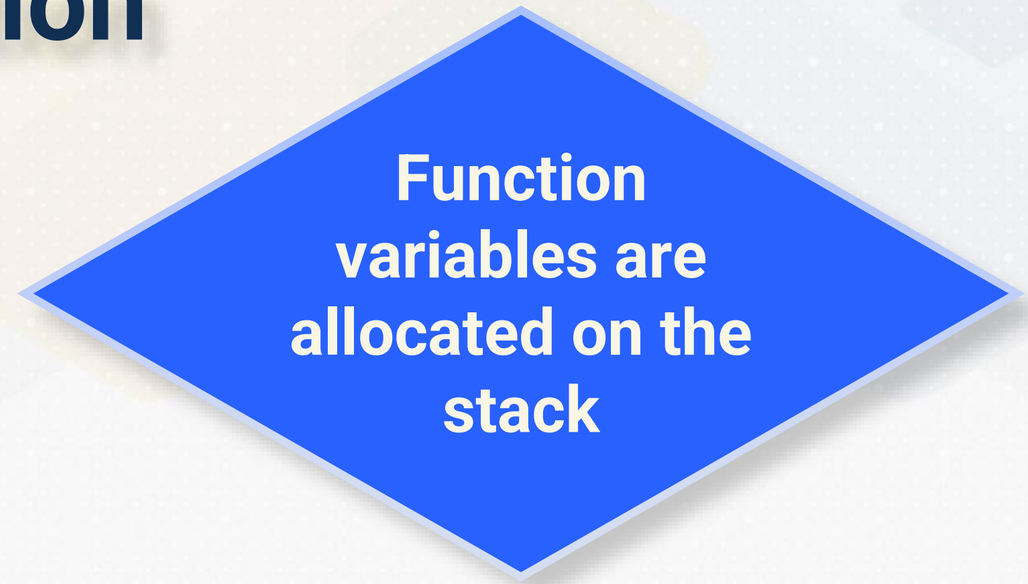


Heap-allocated variables can live beyond the scope in which they were created

Large pool of memory used for dynamic memory allocation

Allows flexible data management

Stack Allocation



Stack Allocation

Function variables are allocated on the stack

Lifetime limited to the scope of the block

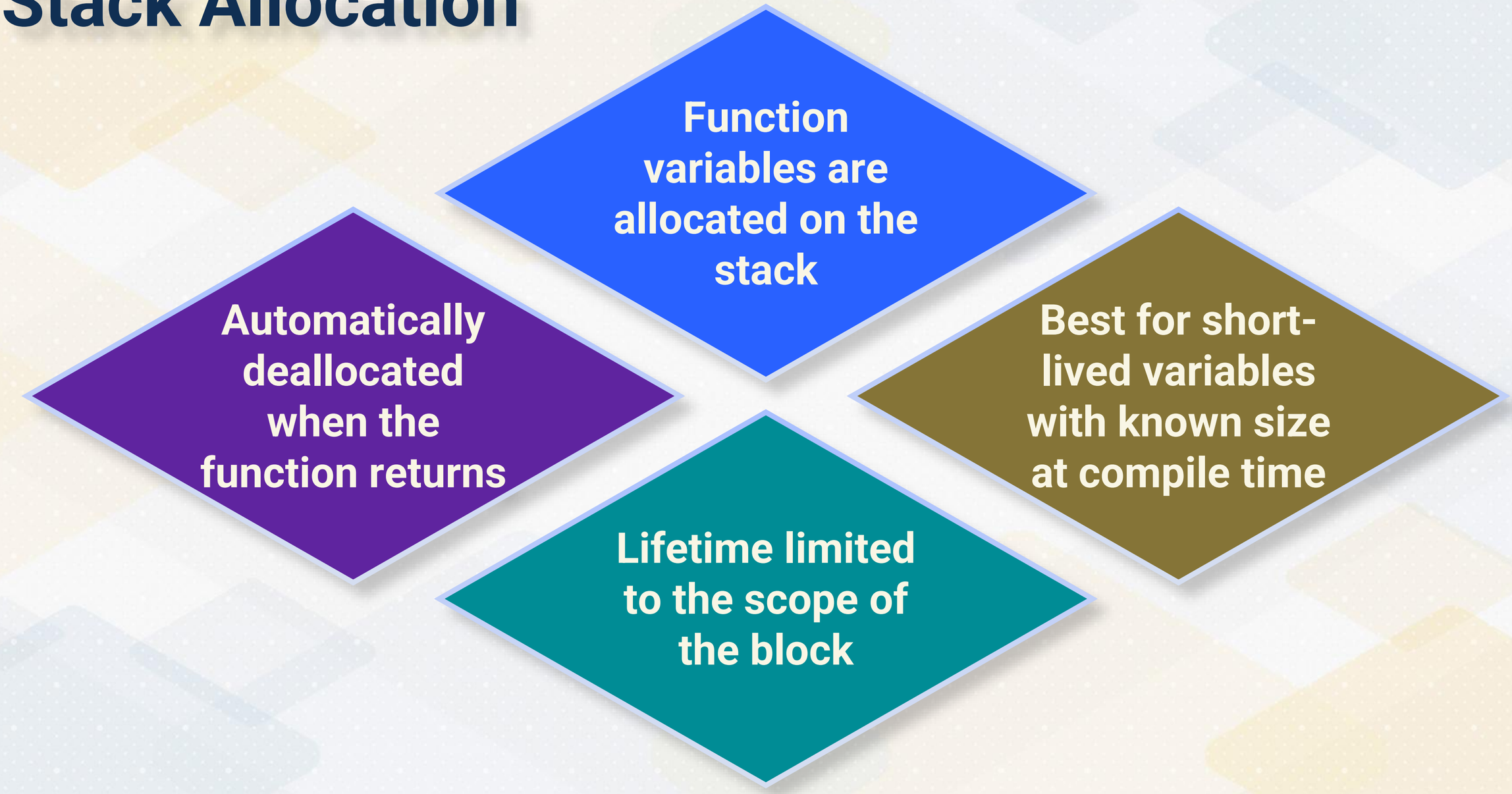
Stack Allocation

Function variables are allocated on the stack

Automatically deallocated when the function returns

Lifetime limited to the scope of the block

Stack Allocation



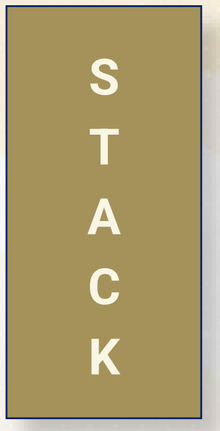
Heap-Based Allocation

```
// Create and returns a smart pointer to a vector on the heap
std::unique_ptr<std::vector<int>> createVectorOnHeap() {
    return std::make_unique<std::vector<int>>(
     std::initializer_list<int>{5, 6, 7, 8});
}
int main() {
    auto heapVector = createVectorOnHeap();
    // heapVector now safely manages the lifetime of the vector
    // Safely accessing the vector
    for (int element : *heapVector) {
        std::cout << element << " ”;
    }
    // The vector's memory is automatically cleaned up here
}
```

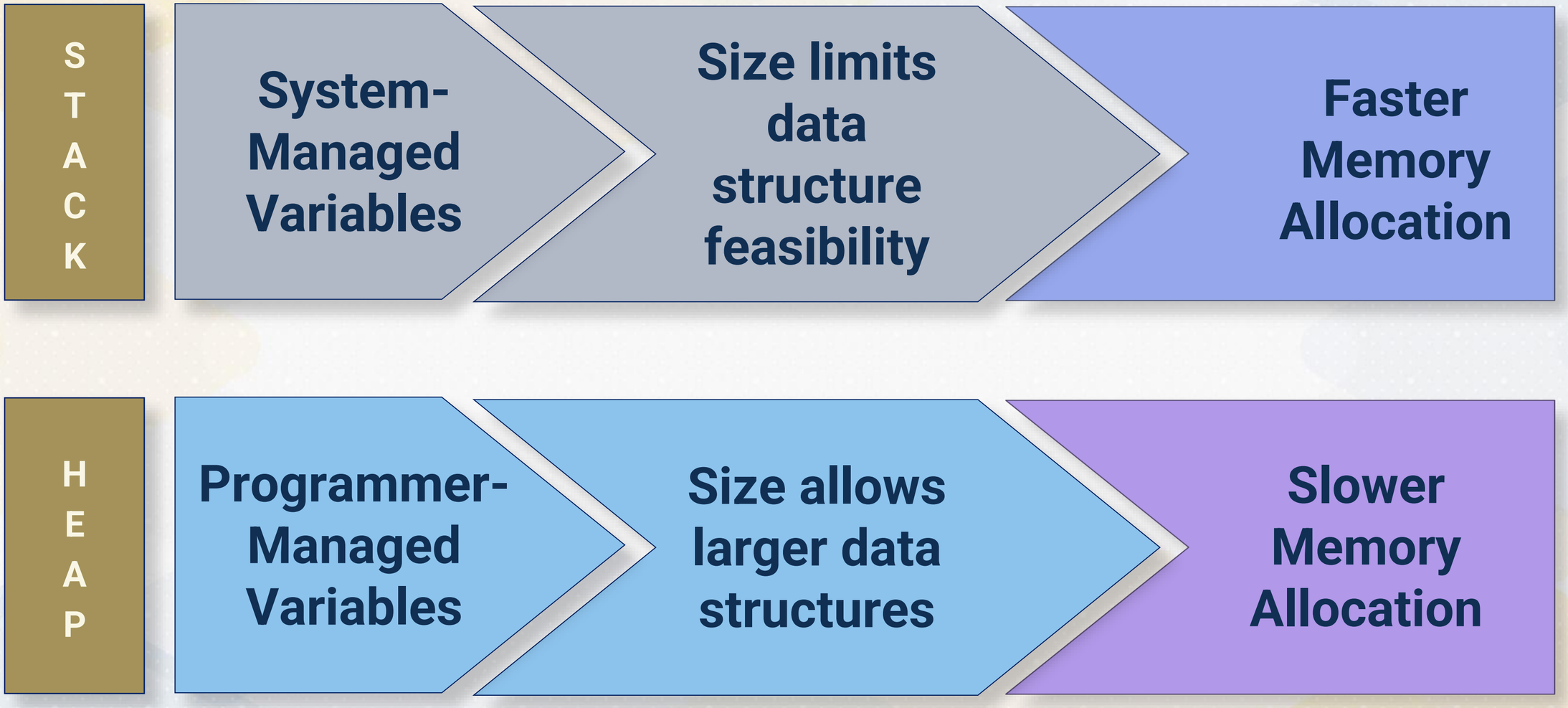
Stack-Based Allocation

```
// Attempts to return a reference to a vector allocated on the stack
const std::vector<int> &createVectorOnStack() {
    std::vector<int> vec = {1, 2, 3, 4};
    // Allocated on the stack
    return vec;
    // Warning: Returning reference to local/temporary object here
}
int main() {
    const auto &myVec = createVectorOnStack();
    // Undefined behavior: myVec refers to a destroyed vector
    // Attempting to use myVec will lead to undefined behavior
    for (int element : myVec) {
        std::cout << element << " ";
    }
}
```

Heap vs Stack



Heap vs Stack



Smart Field and Tuple Classes

Smart Field and Tuple Classes

Copy Semantics in C++

Smart Field and Tuple Classes

Copy Semantics in C++

Heap vs Stack

Page Class

Serialization and Deserialization

Page Class

Page Class

Serialization and Deserialization

Persistence

Persistence

Persistence

Data Managed In-memory

Data Lost on Program Termination

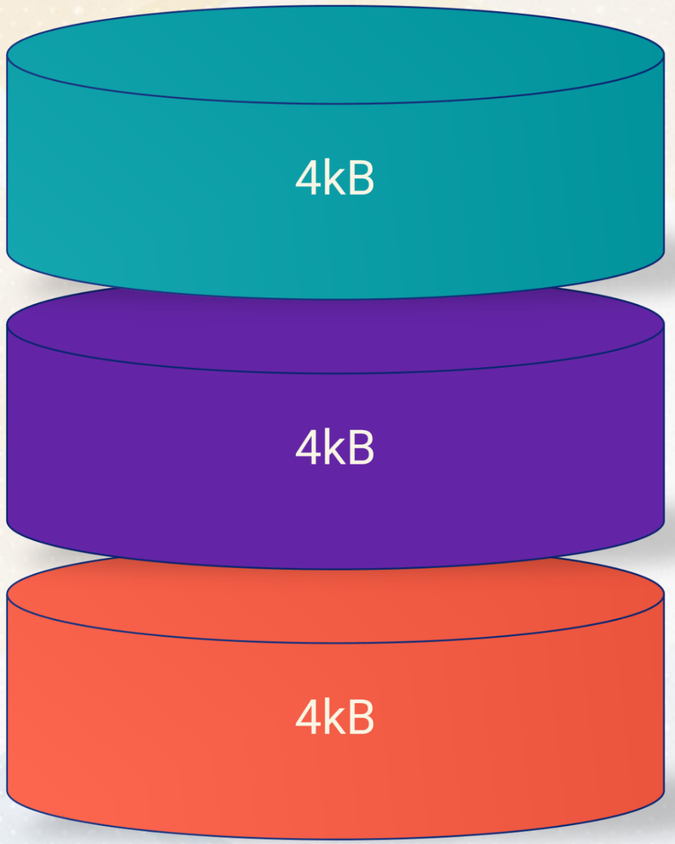
Persistence

Data Managed In-memory

Data Lost on Program Termination

Persist Data Across Sessions

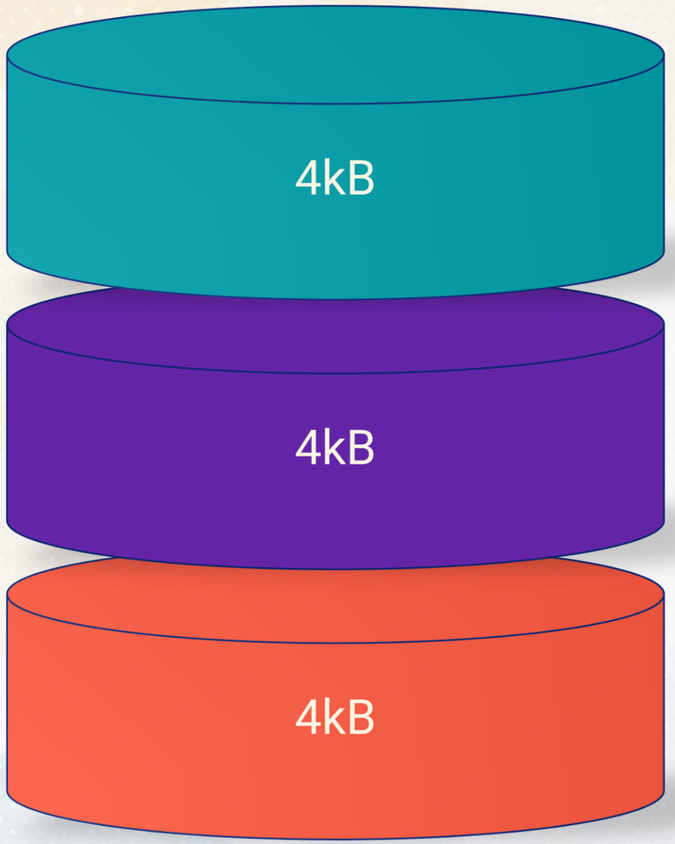
Disks and Pages



Basic unit of disk storage

```
class Page {
public:
  size_t used_size = 0;
  std::vector<std::unique_ptr<Tuple>> tuples;
};
```

Page Class



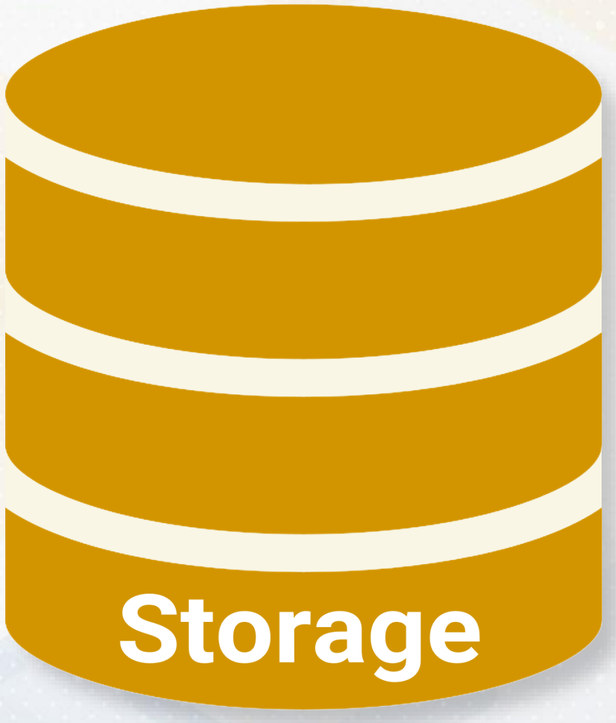
Dynamic Tuple Management

Maintains Page Capacity

```
bool addTuple(std::unique_ptr<Tuple> tuple) {
  size_t tuple_size = // Calculated size;
  if (used_size + tuple_size > PAGE_SIZE)
            return false;
  tuples.push_back(std::move(tuple));
  used_size += tuple_size;
  return true;
}
```

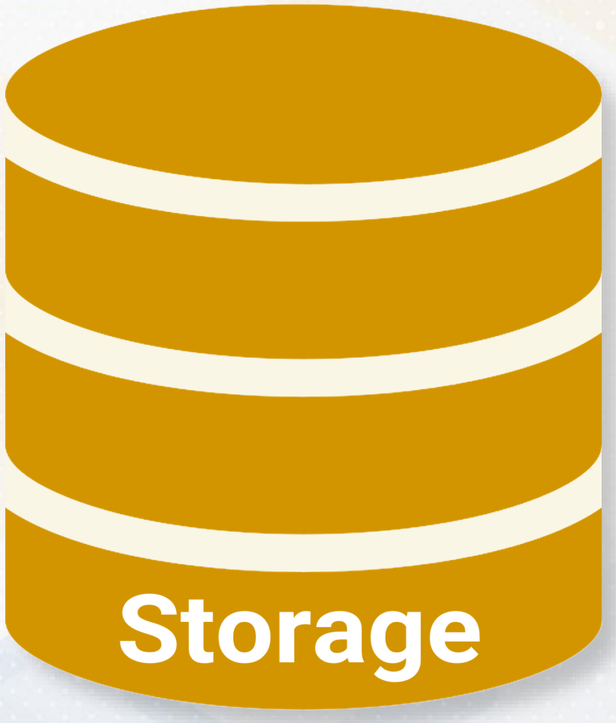
Serialization

Serialization and Deserialization




```
void write(const std::string &filename) const {
    // Logic to write in-memory tuples to
    // a format suitable for disk storage
}
void read(const std::string &filename) {
    // Logic to read a page from disk
    // and reconstruct in-memory tuples
}
```

Serialization and Deserialization



Serialization — Converting In-Memory Data into Disk Format

Deserialization — Loading Pages from Disk into Memory

```
void write(const std::string &filename) const {
    // Logic to write in-memory tuples to
    // a format suitable for disk storage
}
void read(const std::string &filename) {
    // Logic to read a page from disk
    // and reconstruct in-memory tuples
}
```

Serialization of Page

Page Class Serialization Process

Write the number of tuples in Page

Loop through each tuple and serialize its data

```
void write(const std::string &filename) const {
  std::ofstream out(filename);
  // First write the number of tuples.
  size_t numTuples = tuples.size();
  out.write(reinterpret_cast<const char *>(&numTuples), sizeof(numTuples));
  ...
}
```

Serialization of Tuple

Page Class Serialization Process

Write the number of tuples in Page

Loop through each tuple and serialize its data

Record how many fields each tuple contains

Field serialization involves: *Field Type*, *Field Length*, and *Field Data*

Serialization of Field

```
// Then write each tuple.
for (const auto &tuple : tuples) {
  // Write the number of fields in the tuple.
  size_t numFields = tuple->fields.size();
  out.write(reinterpret_cast<const char *>(&numFields), sizeof(numFields));

  // Then write each field.
  for (const auto &field : tuple->fields) {
    out.write(reinterpret_cast<const char *>(&field->type), sizeof(field->type));
    out.write(reinterpret_cast<const char *>(&field->data_length), sizeof(..));
    out.write(field->data.get(), field->data_length);
  }
}
```

Deserialization

Deserialization of Page

Deserialization allows BuzzDB to load pages stored on disk

Deserialization reconstructs in-memory pages

```
// Read this page from a file.
void read(const std::string &filename) {
  std::ifstream in(filename);
  // First read the number of tuples.
  size_t numTuples;
  in.read(reinterpret_cast<char *>(&numTuples), sizeof(numTuples));
}
```

Deserialization of Tuple

Identify the number of fields in a tuple

```
// Read the number of fields in the tuple.
size_t numFields;
in.read(reinterpret_cast<char *>(&numFields), sizeof(numFields));


// Then read each tuple.
for (size_t i = 0; i < numTuples; ++i) {
  auto tuple = std::make_unique<Tuple>();
  // Read the number of fields in the tuple.
  size_t numFields;
  in.read(reinterpret_cast<char *>(&numFields), sizeof(numFields));
```

Deserialization of Field

- Reading the field type and its length
- Reading the data
- Reconstructing the field in memory

```
// Then read each field.
for (size_t j = 0; j < numFields; ++j) {
    // Read the type of the field.
    FieldType type;
    in.read(reinterpret_cast<char *>(&type), sizeof(type));
    // Read the length of the field.
    size_t data_length;
    in.read(reinterpret_cast<char *>(&data_length), sizeof(data_length));
    // Then read the field data.
    std::unique_ptr<char[]> data(new char[data_length]);
    in.read(data.get(), data_length);
```

Deserialization

Tuple Restoration

fields are assembled back into their respective tuples based on their type, whether integer, float, or string

```
// Add the field to the tuple.
  switch (type) {
    ...
    case STRING: {
       char *val = reinterpret_cast<char *>(data.get());
       auto field = std::make_unique<Field>(std::string(val, data_length));
       tuple->addField(std::move(field));
       break;
    }
 }
```

Deserialization

Tuple Restoration

Deserialization process is complete

```
std::cout << "Tuple " << (i + 1) << " :: ";
tuple->print();
```

Using Page Class in BuzzDB

Tuples pushed to a Page

```
class BuzzDB {
    Page page;
    void insert(int key, int value) {
        page.addTuple(std::move(newTuple));
    }
};
int main() {
    // Serialize page to disk
    db.page.write(filename);
    // Deserialize page from disk
    Page page2;
    page2.read(filename);
}
```

Page Class

Page Class

Serialization and Deserialization

Simplifying Page Serialization

Static Methods in C++

Simplifying Page Serialization

Simplifying Page Serialization

Static Methods in C++

Simplifying Page Serialization

```
void Field::serialize(std::ofstream &out) {
  out << type << ' ' << data_length << ' ';
  if (type == STRING) {
    out << data.get() << ' ';
  } else if (type == INT) {
    out << *reinterpret_cast<int *>(data.get()) << ' ';
  } else if (type == FLOAT) {
    out << *reinterpret_cast<float *>(data.get()) << ' ';
  }
}
```

Simplifying Page Serialization

Field Class Serialization

```
void Field::serialize(std::ofstream &out) {
  out << type << ' ' << data_length << ' ';
  if (type == STRING) {
    out << data.get() << ' ';
  } else if (type == INT) {
    out << *reinterpret_cast<int *>(data.get()) << ' ';
  } else if (type == FLOAT) {
    out << *reinterpret_cast<float *>(data.get()) << ' ';
  }
}
```

Simplifying Page Serialization

```
void Tuple::serialize(std::ofstream &out) {
  out << fields.size() << ' ';
  for (auto &field : fields) {
    field->serialize(out);
  }
}
```

Simplifying Page Serialization

Tuple Class Serialization

```
void Tuple::serialize(std::ofstream &out) {
  out << fields.size() << ' ';
  for (auto &field : fields) {
    field->serialize(out);
  }
}
```

Simplifying Page Serialization

Page Serialization Simplification

```
void Page::write(const std::string &filename) const {
  std::ofstream out(filename);
  // Serialize the number of tuples in the page
  out << tuples.size() << '\n';
  // Loop through each tuple and serialize its data
  for (auto &tuple : tuples) {
    tuple->serialize(out);
    out << '\n'; // Delimiter for each tuple
  }
  out.close();
}
```

Simplifying Page Serialization

Page Serialization Simplification

Number of Tuples

```
void Page::write(const std::string &filename) const {
  std::ofstream out(filename);
  // Serialize the number of tuples in the page
  out << tuples.size() << '\n';
  // Loop through each tuple and serialize its data
  for (auto &tuple : tuples) {
    tuple->serialize(out);
    out << '\n'; // Delimiter for each tuple
  }
  out.close();
}
```

Simplifying Page Serialization

Page Serialization Simplification

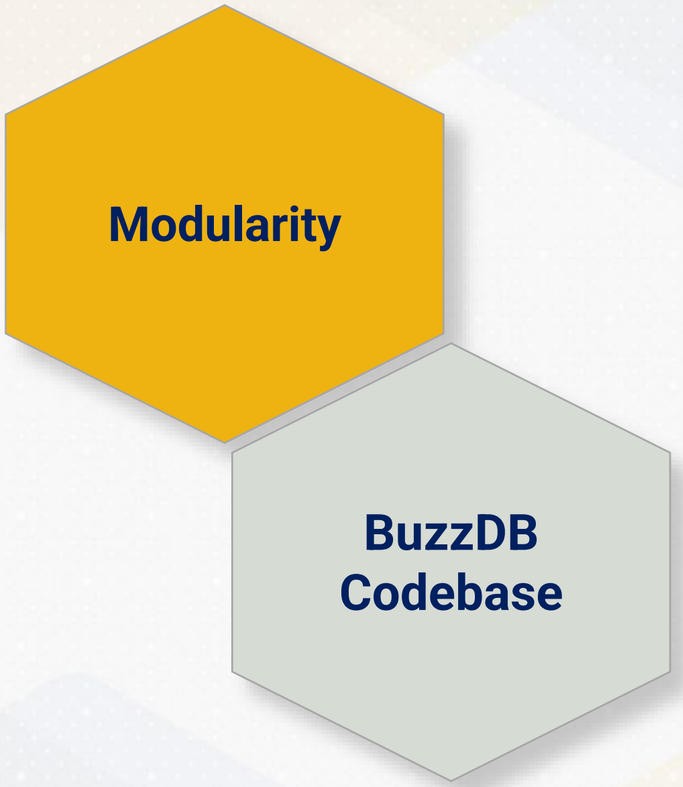
Number of Tuples

Loop through Tuples

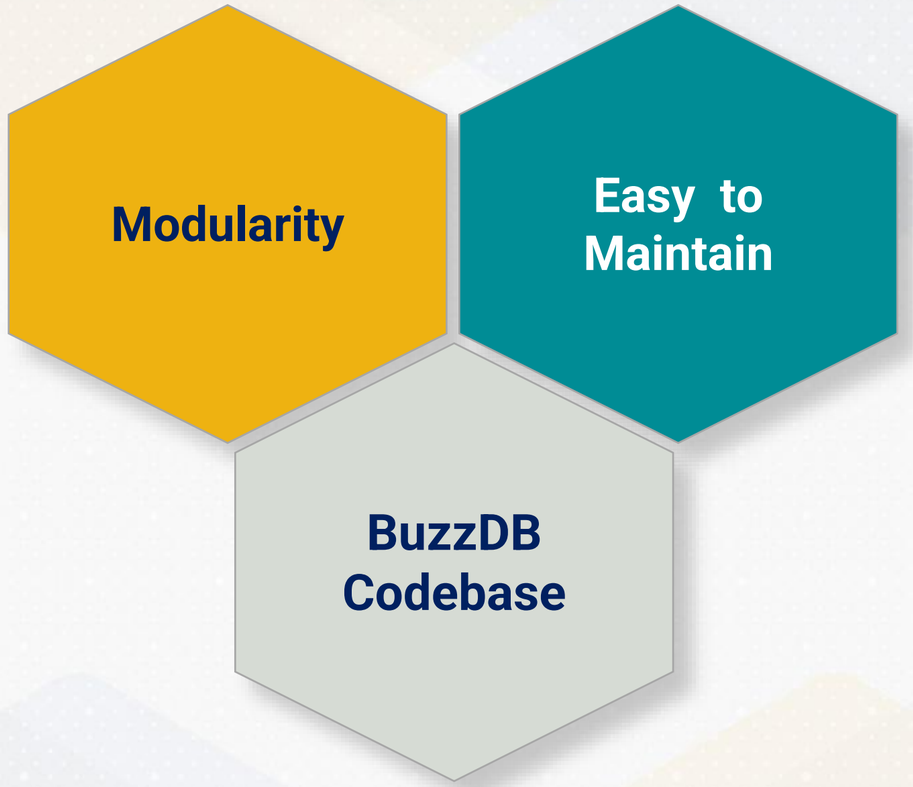
```
void Page::write(const std::string &filename) const {
  std::ofstream out(filename);
  // Serialize the number of tuples in the page
  out << tuples.size() << '\n';
  // Loop through each tuple and serialize its data
  for (auto &tuple : tuples) {
    tuple->serialize(out);
    out << '\n'; // Delimiter for each tuple
  }
  out.close();
}
```

Modular Approach to Serialization

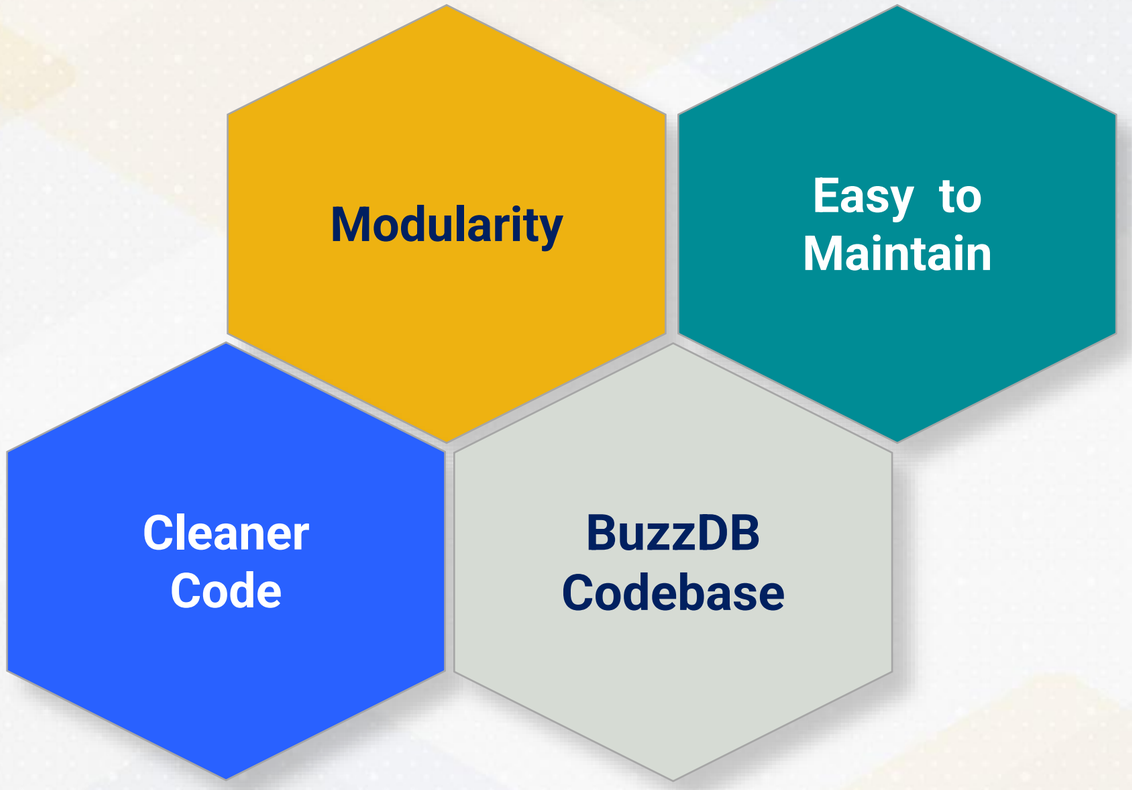
Modular Approach to Serialization



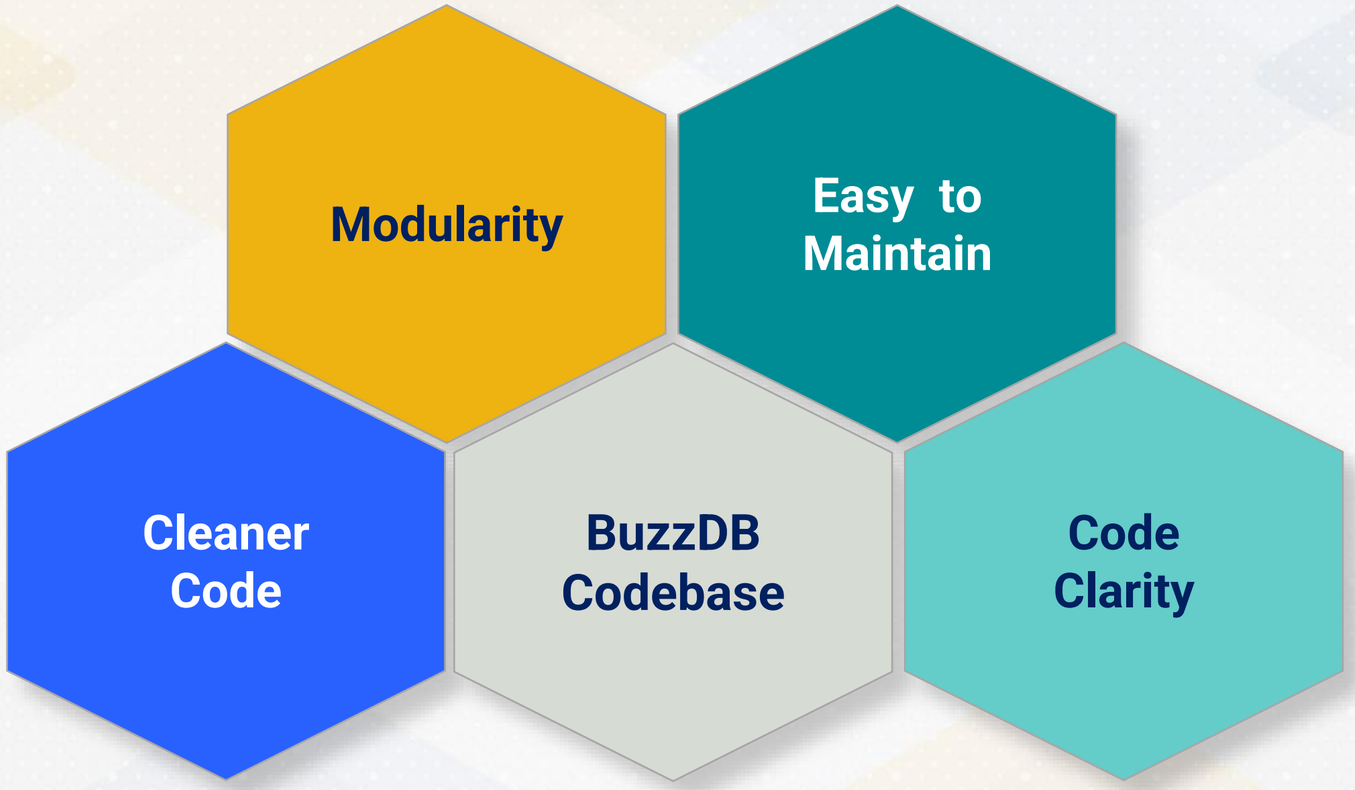
Modular Approach to Serialization



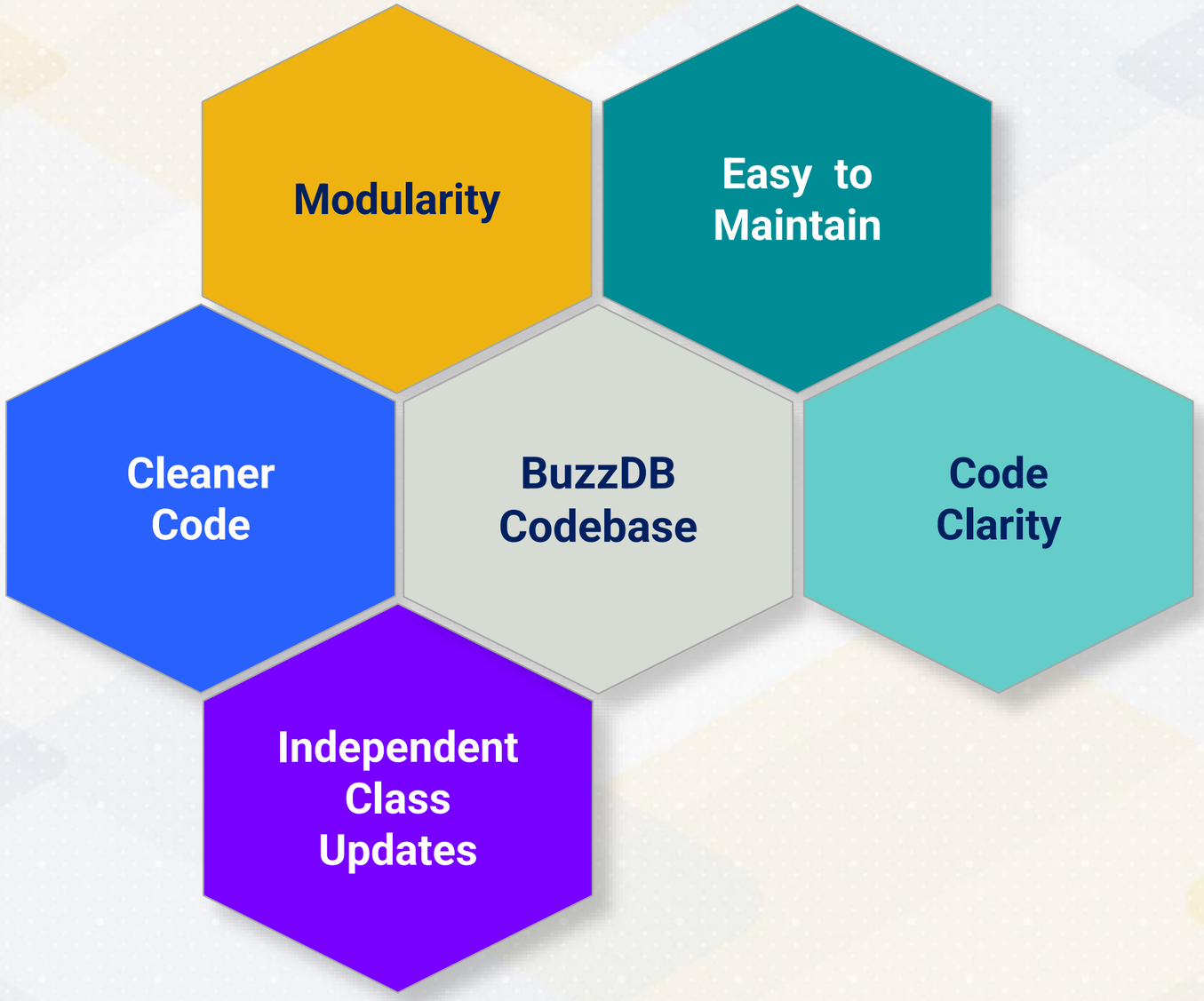
Modular Approach to Serialization



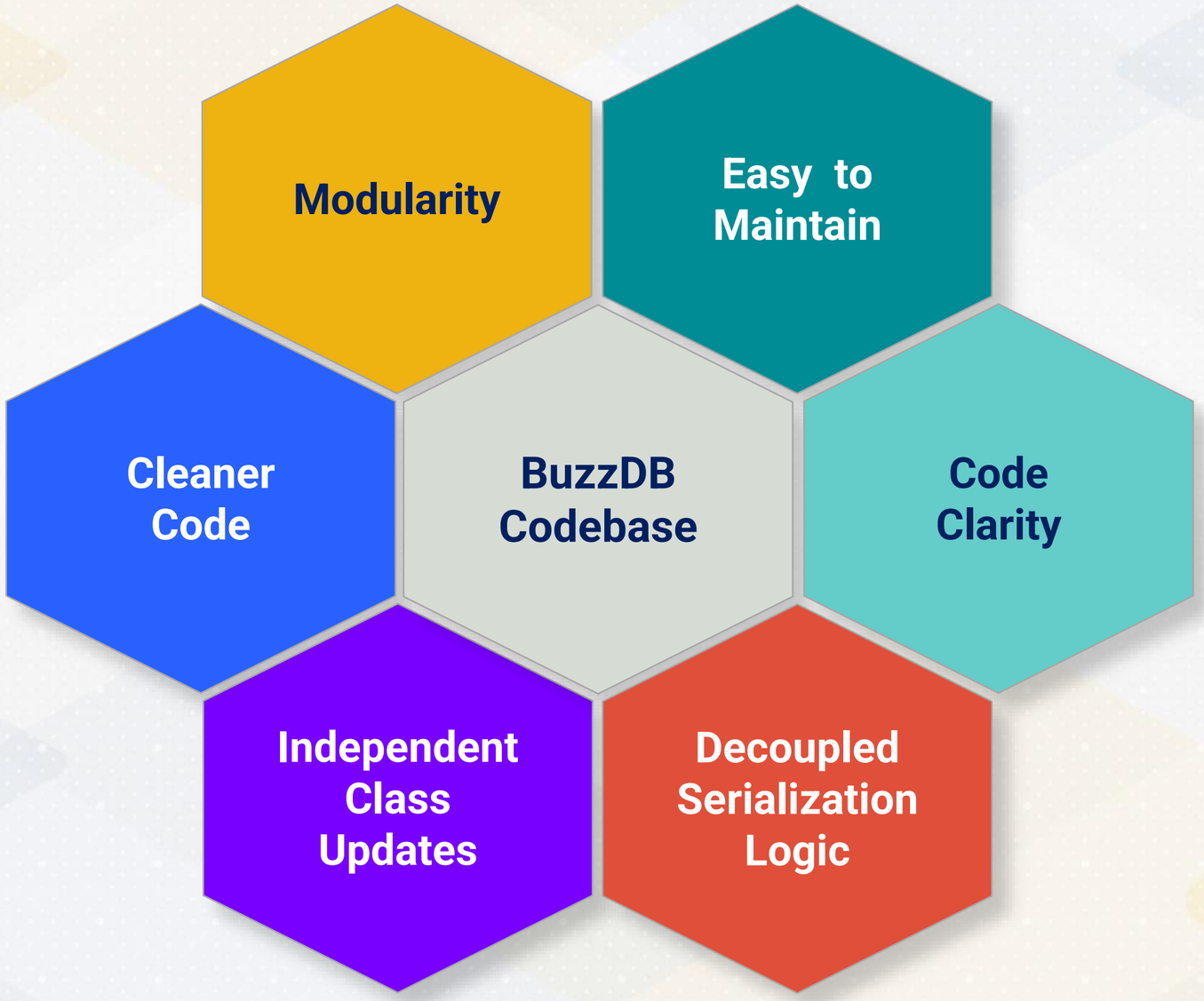
Modular Approach to Serialization



Modular Approach to Serialization



Modular Approach to Serialization



Static Method for Deserialization

Instance Method for Deserialization

```
// Serialize to disk
db.page.write(filename);


// Deserialize from disk
Page page2;
page2.read(filename);
```

Instance Method for Deserialization

Deserialization Steps

Page Object Creation

Page Data Loading

```
// Serialize to disk
db.page.write(filename);


// Deserialize from disk
Page page2;
page2.read(filename);
```

Static vs Instance Methods in C++

Static Methods

- Pertains to Whole Class
- Invoke without Class Object Creation

Instance Methods

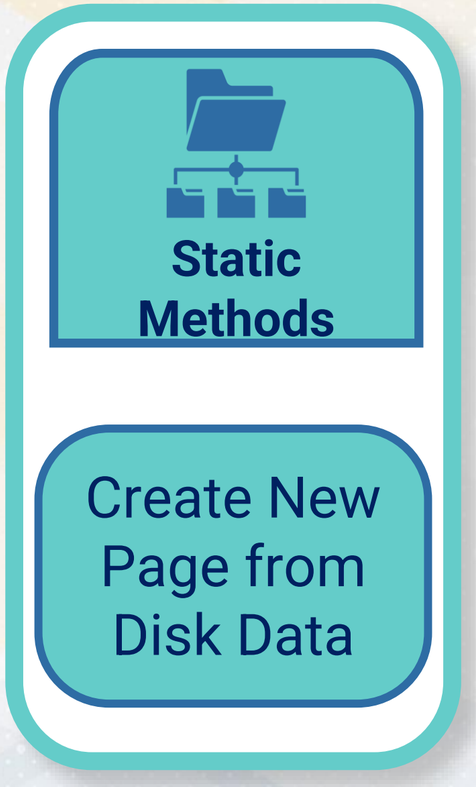
- Operate/Modify Data
- Applies to Specific Class Instances

Page::deserialize(filename)

- Merge Creation + Loading
- Direct Tuple Loading
- No Temporary Page Object

```
auto loadedPage = Page::deserialize(filename);


// Deserialize from disk
// Page page2;
// page2.read(filename);
```

Static and Instance Methods




```
// Read this page from a file.
static std::unique_ptr<Page>
deserialize(const std::string &filename) {
  std::ifstream in(filename);
  auto page = std::make_unique<Page>();
  ...
  return page;
}
```

Simplifying Page Serialization

Simplifying Page Serialization

Static Methods in C++

Tuple Deletion

Data Consistency

Tuple Deletion

Tuple Deletion

Data Consistency

Tuple Deletion

Tuple Deletion Process

- deleteTuple method
- Check vector bounds
- Update page size
- Remove tuple from vector

```
void Page::deleteTuple(size_t index) {
  if (index >= tuples.size()) {
    std::cout << "Tuple index out of range. ";
    return;
  }
  used_size -= tuples[index]->getSize();
  tuples.erase(tuples.begin() + index);
}
```

Tuple Deletion

Logic introduced to delete tuples

```
// Skip deleting tuples only once every hundred tuples
if (tuple_insertion_attempt_counter % 100 != 0) {
  page.deleteTuple(0);
}
```

Tuple Deletion

Logic introduced to delete tuples

Simulate Data Lifestyle

```
// Skip deleting tuples only once every hundred tuples
if (tuple_insertion_attempt_counter % 100 != 0) {
  page.deleteTuple(0);
}
```

Tuple Deletion

Logic introduced to delete tuples

Simulate Data Lifestyle

Skip Deletion Periodically

```
// Skip deleting tuples only once every hundred tuples
if (tuple_insertion_attempt_counter % 100 != 0) {
  page.deleteTuple(0);
}
```

Tuple Deletion

Logic introduced to delete tuples

Simulate Data Lifestyle

Skip Deletion Periodically

Database Cleanup

```
// Skip deleting tuples only once every hundred tuples
if (tuple_insertion_attempt_counter % 100 != 0) {
  page.deleteTuple(0);
}
```

Data Inconsistency

In-Memory Deletion

```
// Serialize to disk
db.page.write(filename);
// Deserialize from disk
auto loadedPage = Page::deserialize(filename);
// PROBLEM: Deletion only in memory, not on disk
loadedPage->deleteTuple(0);
// Deserialize again from disk -- page unchanged
auto loadedPage2 = Page::deserialize(filename);
```

Data Inconsistency

In-Memory Deletion

Changes do not reach disk

```
// Serialize to disk
db.page.write(filename);
// Deserialize from disk
auto loadedPage = Page::deserialize(filename);
// PROBLEM: Deletion only in memory, not on disk
loadedPage->deleteTuple(0);
// Deserialize again from disk -- page unchanged
auto loadedPage2 = Page::deserialize(filename);
```

Data Inconsistency

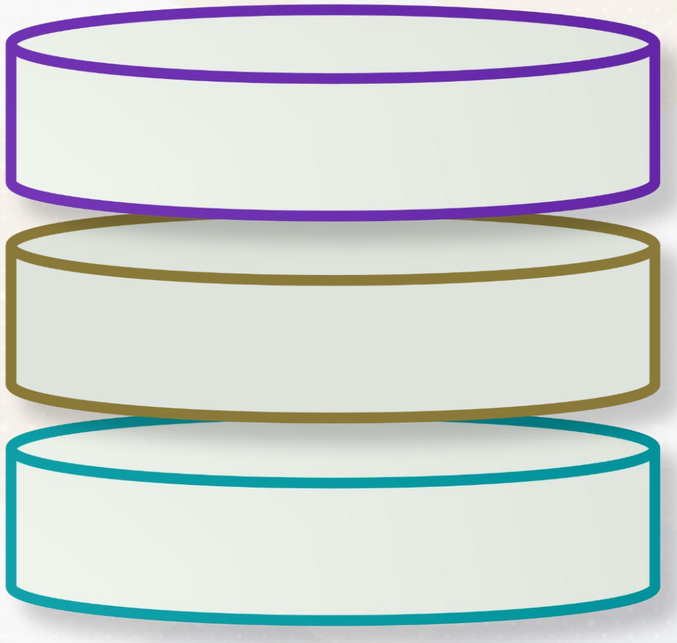
In-Memory Deletion

Changes do not reach disk

loadedPage2 mismatch

```
// Serialize to disk
db.page.write(filename);
// Deserialize from disk
auto loadedPage = Page::deserialize(filename);
// PROBLEM: Deletion only in memory, not on disk
loadedPage->deleteTuple(0);
// Deserialize again from disk -- page unchanged
auto loadedPage2 = Page::deserialize(filename);
```

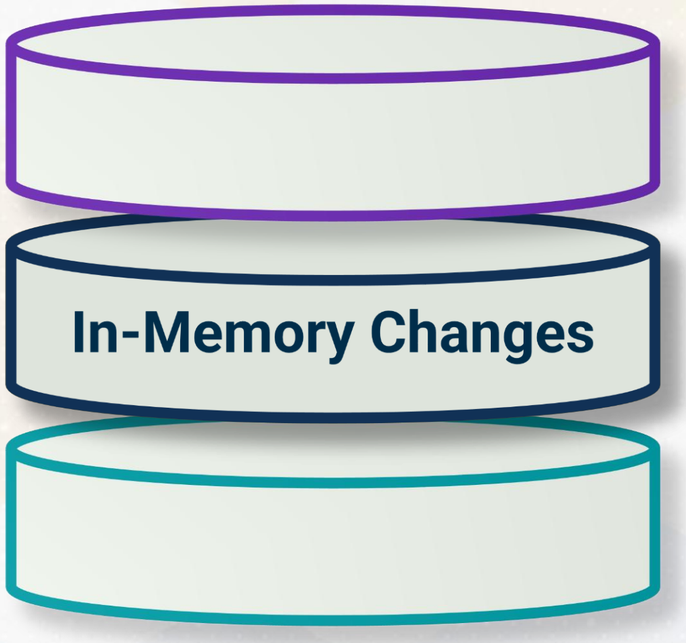
Data Inconsistency



Data Inconsistency Issues

- In-Memory Changes Not Persisted
- Memory and Disk State Discrepancy

Data Inconsistency



Data Inconsistency Issues

- In-Memory Changes Not Persisted
- Memory and Disk State Discrepancy

Data Synchronization


```
std::cout << "Deleting slots 0 and 7 \n";
loadedPage->deleteTuple(0);
loadedPage->deleteTuple(7);

loadedPage->write(filename);
```

Data Synchronization

Modifications reflected during deserialization

```
// Deserialize again from disk -- page is updated this time
auto loadedPage2 = SlottedPage::deserialize(filename);
loadedPage2->print();
```

Tuple Deletion

Tuple Deletion

Data Consistency